

# Companion to Bird & de Moor, Algebra of Programming

## §9. Dynamic Programming

§9.1. Theory

§9.1.1. Example: segmenting a list

- (9.1.1a)
- $\langle T \rangle = \text{concat}$   
 $T=[\text{nil}, \text{cat}]$  takes a non-empty first list, so  $T^\circ$  cuts a non-empty prefix off a list every way, and folding with  $T$  flattens a list of segments
  - $\langle h \rangle = \text{list}(\text{sum})$   
 $h=[\text{nil}, \text{cons}(\text{sum}\times 1)]$  puts a segment's sum in front, so folding with  $h$  sums every segment
  - $H = \text{partition list}(\text{sum})$   
 $H$  segments a list every way, then sums each segment; with  $R$  comparing the largest entry,  $M$  cuts the list so that the largest segment sum is as small as possible

(9.1.1b)

|   | <b>x</b>          | <b>h(x)</b>  |
|---|-------------------|--------------|
| 1 | $l(())$           | $[\ ]$       |
| 2 | $r([c], [])$      | $[c]$        |
| 3 | $r([a, b], [c])$  | $[a + b, c]$ |
| 4 | $r([a], [b + c])$ | $[a, b + c]$ |

- (9.1.1c)
- $H([a, b, c], ys) \iff ys = [a + b + c] \vee ys = [a, b + c] \vee ys = [a + b, c] \vee ys = [a, b, c]$   
 $H$  sends  $[a, b, c]$  to the sums of its four segmentations and nothing else
  - $T^\circ F(H) h = H$   
cut once, solve the rest, put the pieces back together

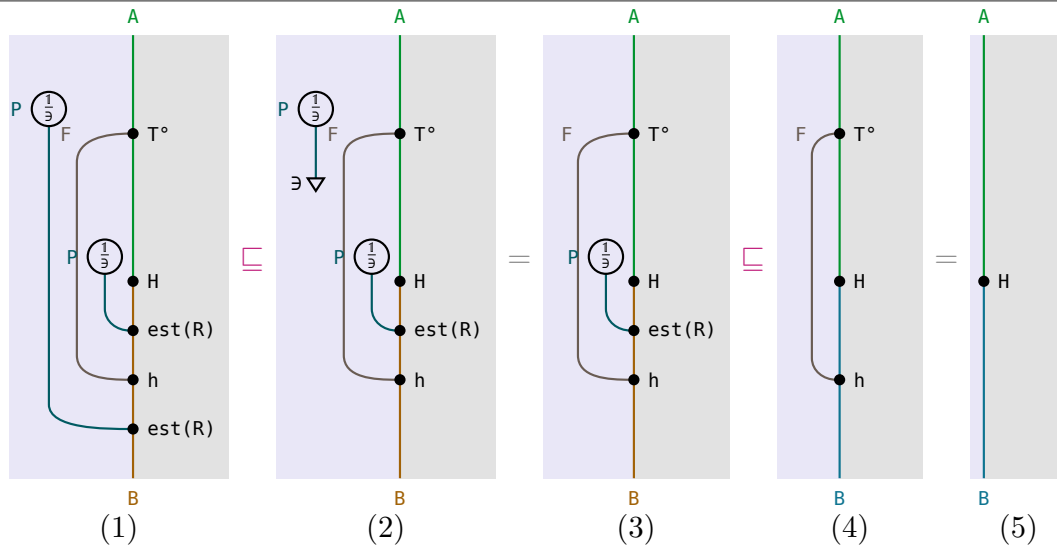
(9.1.1d)

|   | <b>name</b> | <b>type</b> | <b>definition</b>                                        | <b>meaning</b>                                                           |
|---|-------------|-------------|----------------------------------------------------------|--------------------------------------------------------------------------|
| 1 | <b>H</b>    | <b>A→B</b>  | $H \triangleq \langle T \rangle^\circ \langle h \rangle$ | take the input apart by the T-decompositions, and put it together by $h$ |
| 2 | <b>M</b>    | <b>A→B</b>  | $M \triangleq \frac{H}{\exists} \text{ est}(R)$          | the problem to be solved: an answer of $H$ that is best under $R$        |

| <b>Map(h) <math>\wedge</math> h Pres <math>R^\circ \wedge R^\circ R^\circ \sqsubseteq R^\circ \implies (\mu X : \frac{T^\circ}{\exists} P(F(X) h) \text{ est}(R)) \sqsubseteq \frac{H}{\exists} \text{ est}(R)</math></b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |  |  |  | Theorem 9.1 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|-------------|
| <b>every answer the recursion returns is an optimal one</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |  |  |  |             |
| <ul style="list-style-type: none"><li>• <math>\frac{T^\circ}{\exists}</math> takes the input apart one step every way; <math>F(X)</math> solves each part by the recursion <math>X</math>; <math>h</math> assembles each candidate; <math>\text{est}(R)</math> keeps a best one</li><li>• <math>\sqsubseteq M</math>: that never misses the optimum, though all candidates are never listed as <math>H</math> would</li><li>• <math>h</math> monotonic on <math>R</math>: a better part never makes the whole worse</li><li>• <math>\sqsubseteq</math>, not <math>=</math>: some optima may not be returned; that the recursion returns anything at all needs <math>T^\circ</math> to stop taking apart — dynamic programming theorem</li></ul> |  |  |  |             |

$$\frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R) \sqsubseteq H \quad (9.2)$$

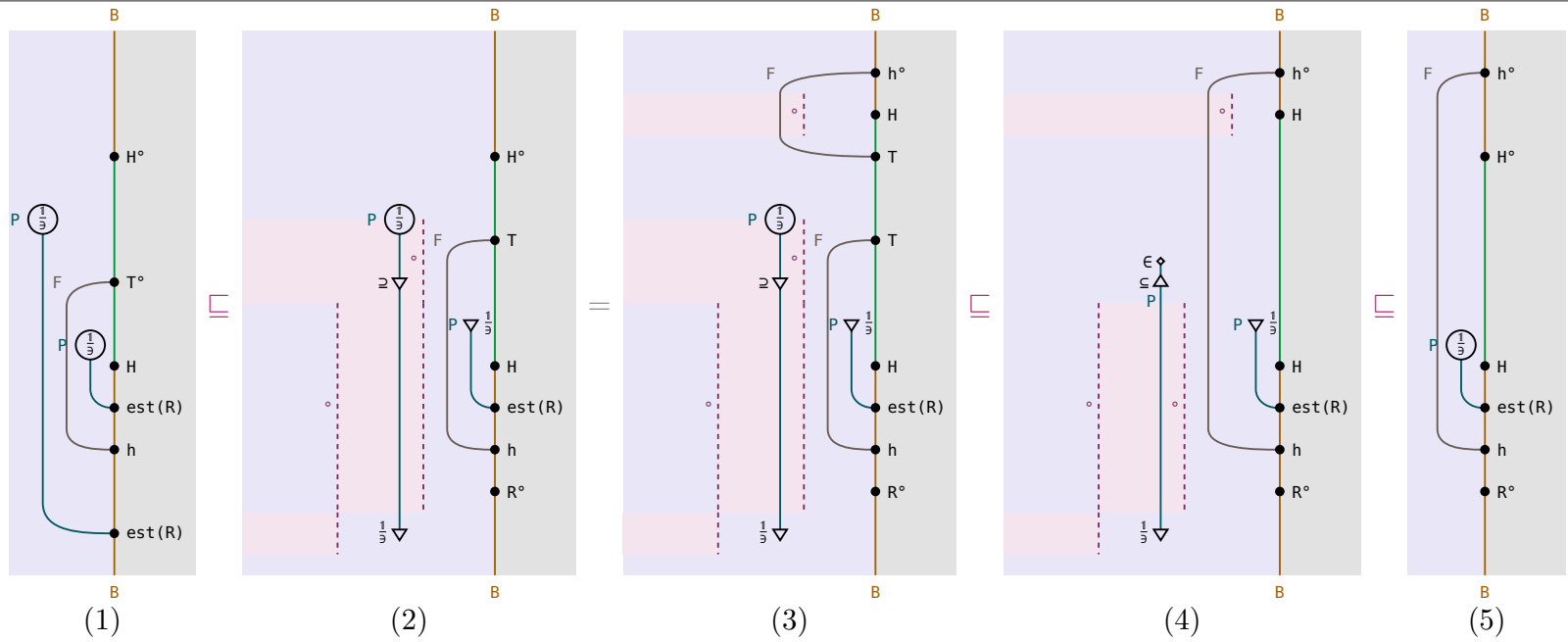
taking the input apart every way  $T$  allows ( $\frac{T^\circ}{\exists}$ ), solving each part by  $M$  and keeping an optimum ( $P(F(M)h) \text{est}(R)$ )  
returns only what  $H$  returns



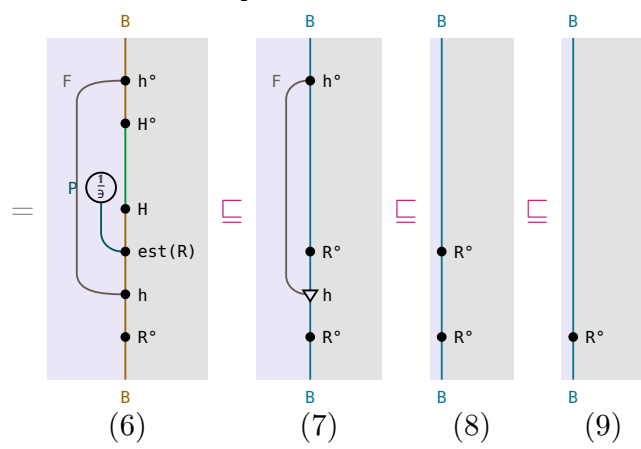
|   |                                                                                    |                                                 |
|---|------------------------------------------------------------------------------------|-------------------------------------------------|
| 1 | $\frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R)$    | $P(S) \text{est}(R) \sqsubseteq \exists S$      |
| 2 | $\sqsubseteq \frac{T^\circ}{\exists} \exists F(\frac{H}{\exists} \text{est}(R)) h$ | $\frac{R}{\exists} \exists = R$                 |
| 3 | $= T^\circ F(\frac{H}{\exists} \text{est}(R)) h$                                   | $\frac{S}{\exists} \text{est}(R) \sqsubseteq S$ |
| 4 | $\sqsubseteq T^\circ F(H) h$                                                       | $T^\circ F(H) h = H$                            |
| 5 | $= H$                                                                              |                                                 |

$$\text{Map}(h) \wedge h \text{ Pres } R^\circ \wedge R^\circ R^\circ \sqsubseteq R^\circ \implies H^\circ \frac{T^\circ}{3} P(F(\frac{H}{3} \text{ est}(R)) h) \text{ est}(R) \sqsubseteq R^\circ \quad (9.3)$$

for every  $b$  that  $H$  returns from an input, the step  $\frac{T^\circ}{3} P(F(M)h) \text{ est}(R)$  returns from that input only  $b'$  with  $R$  relating  $b'$  to  $b$



|   |                                                                                                           |                                                                  |
|---|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| 1 | $H^\circ \frac{T^\circ}{3} P(F(\frac{H}{3} \text{ est}(R)) h) \text{ est}(R)$                             | $P(S) \text{ est}(R) \sqsubseteq \epsilon \setminus (S R^\circ)$ |
| 2 | $\sqsubseteq H^\circ \frac{T^\circ}{3} (\epsilon \setminus (F(\frac{H}{3} \text{ est}(R)) h R^\circ))$    | $H^\circ = h^\circ F(H^\circ) T$                                 |
| 3 | $= h^\circ F(H^\circ) T \frac{T^\circ}{3} (\epsilon \setminus (F(\frac{H}{3} \text{ est}(R)) h R^\circ))$ | $T \frac{T^\circ}{3} \sqsubseteq \epsilon$                       |
| 4 | $\sqsubseteq h^\circ F(H^\circ) \epsilon (\epsilon \setminus (F(\frac{H}{3} \text{ est}(R)) h R^\circ))$  | $S (S \setminus R) \sqsubseteq R$                                |
| 5 | $\sqsubseteq h^\circ F(H^\circ) F(\frac{H}{3} \text{ est}(R)) h R^\circ$                                  |                                                                  |

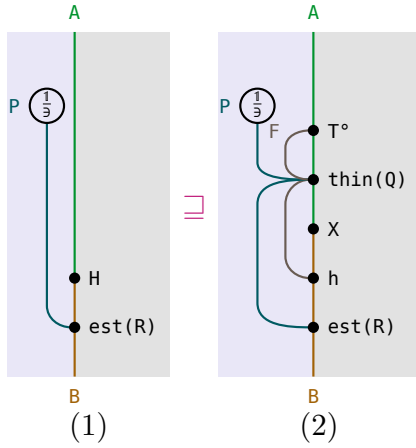


|   |                                                             |                                                                |
|---|-------------------------------------------------------------|----------------------------------------------------------------|
| 6 | $= h^\circ F(H^\circ \frac{H}{3} \text{ est}(R)) h R^\circ$ | $S^\circ \frac{S}{3} \text{ est}(R) \sqsubseteq R^\circ$       |
| 7 | $\sqsubseteq h^\circ F(R^\circ) h R^\circ$                  | $\text{Map}(f)$                                                |
| 8 | $\sqsubseteq R^\circ R^\circ$                               | $\implies f \text{ Pres } R \iff f^\circ F(R) f \sqsubseteq R$ |
| 9 | $\sqsubseteq R^\circ$                                       | $R^\circ R^\circ \sqsubseteq R^\circ$                          |

**Map(h)  $\wedge$  h Pres  $R^\circ \wedge R^\circ R^\circ \subseteq R^\circ \wedge \text{ThinCondition}(T, h, R, Q) \Rightarrow (\mu X : \frac{T^\circ}{S} \text{thin}(Q) P(F(X) h) \text{est}(R)) \subseteq \frac{H}{S} \text{est}(R)$**  Theorem 9.2

an optimum over everything **H** returns is reached by taking the input apart every way **T** allows, dropping the parts that can never win, solving each of the rest and keeping one optimum

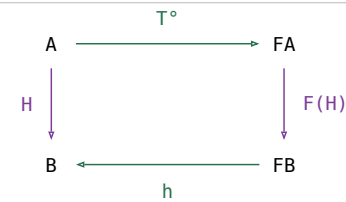
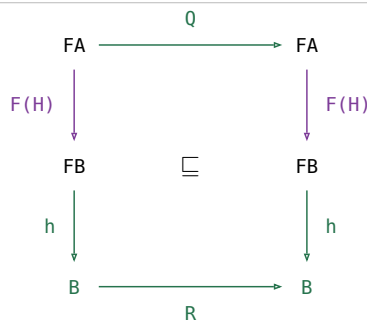
$$H \triangleq (T)^\circ (h) \text{ — } H \triangleq (T)^\circ (h)$$



$$1 \quad \frac{H}{S} \text{est}(R)$$

$$2 \quad \sqsubseteq \frac{T^\circ}{S} \text{thin}(Q) P(F(X) h) \text{est}(R)$$

$QF(H)h \subseteq F(H)hR$ ,  $h$  monotonic on  $R \text{ — } Q \subseteq R \wedge 1 \subseteq Q \wedge R$   
 $R \subseteq R$   
 $\Rightarrow \text{est}(R) = \text{thin}(Q) \text{est}(R), X \subseteq \text{est}(R) \Leftrightarrow X \subseteq \exists$   
 $\wedge \in X \subseteq R^\circ$

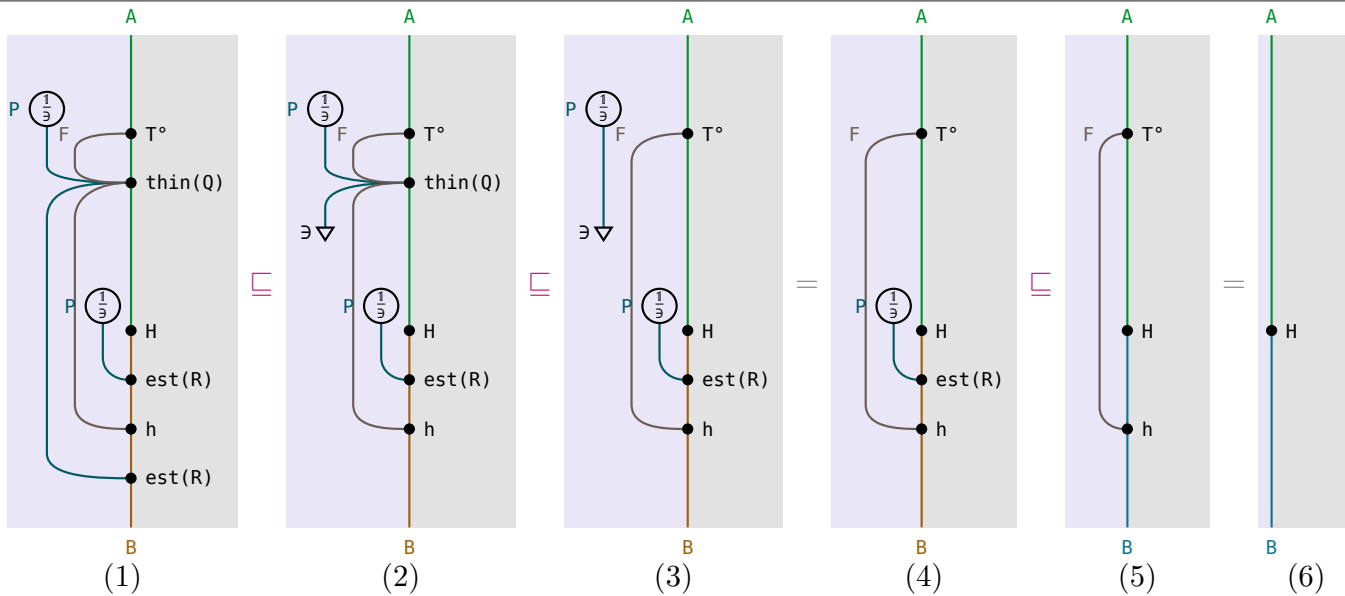


$T^\circ F(H)h = H$ : taking the input apart once by  $T^\circ$ , solving the parts by  $F(H)$  and assembling by  $h$  returns exactly what  $H$  returns

$QF(H)h \subseteq F(H)hR$ : every result of thinning by  $Q$ , solving the parts by  $F(H)$  and assembling by  $h$  is reached by  $R$  from a result of solving by  $F(H)$  and assembling by  $h$  without  $Q$

**$\frac{T^\circ}{S} \text{thin}(Q) P(F(\frac{H}{S} \text{est}(R)) h) \text{est}(R) \subseteq H$**  (9.1.1e)

taking the input apart every way **T** allows ( $\frac{T^\circ}{S}$ ), dropping the parts **thin(Q)** rejects, solving each rest by **M** and keeping an optimum returns only what **H** returns



$$1 \quad \frac{T^\circ}{S} \text{thin}(Q) P(F(\frac{H}{S} \text{est}(R)) h) \text{est}(R)$$

$$2 \quad \sqsubseteq \frac{T^\circ}{S} \text{thin}(Q) \exists F(\frac{H}{S} \text{est}(R)) h$$

$$3 \quad \sqsubseteq \frac{T^\circ}{S} \exists F(\frac{H}{S} \text{est}(R)) h$$

$$4 \quad = T^\circ F(\frac{H}{S} \text{est}(R)) h$$

$$5 \quad \sqsubseteq T^\circ F(H) h$$

$$6 \quad = H$$

$$P(S) \text{est}(R) \subseteq \exists S$$

$$\text{thin}(Q) \exists \subseteq \exists$$

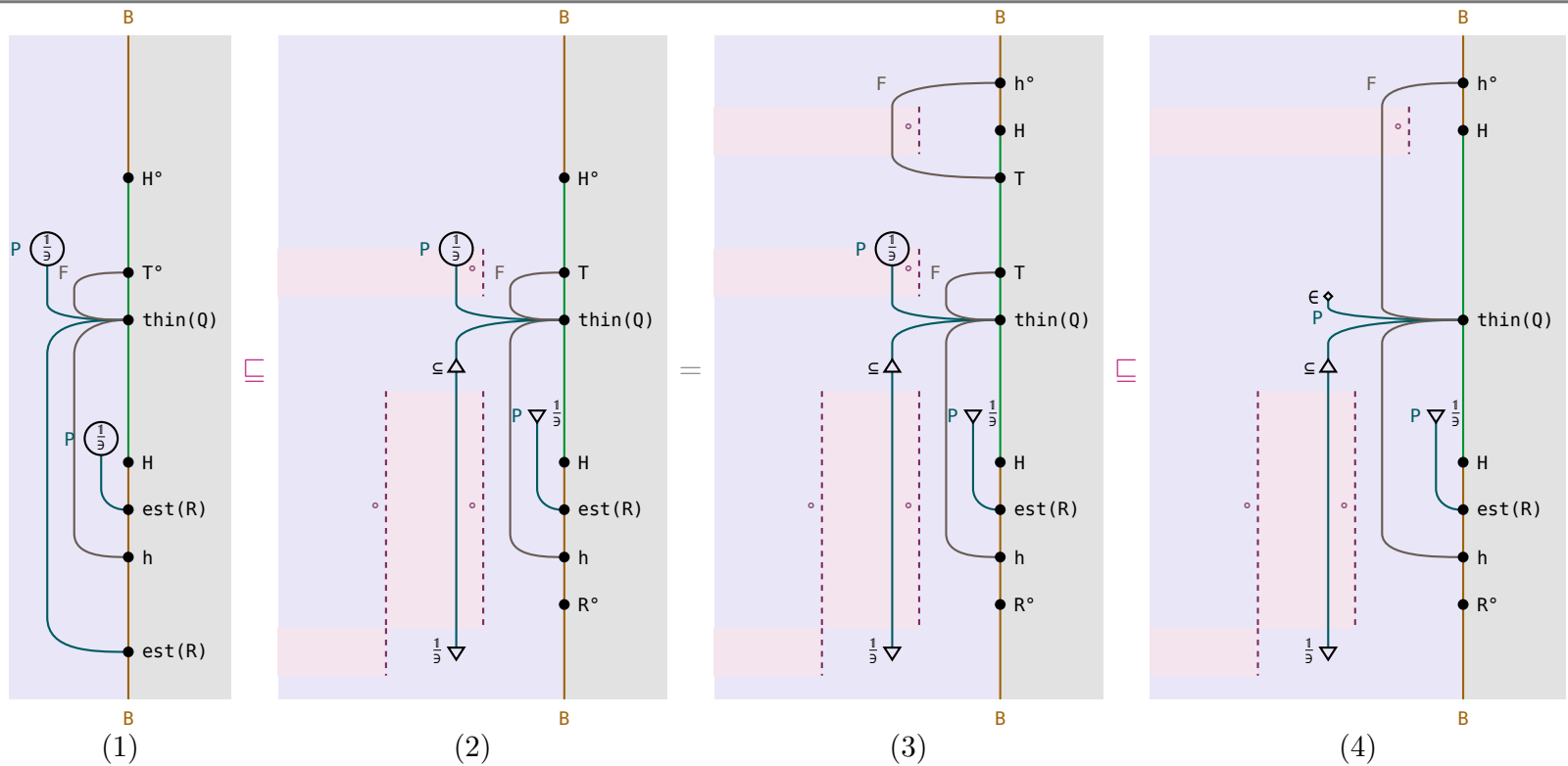
$$\frac{R}{S} \exists = R$$

$$\frac{S}{S} \text{est}(R) \subseteq S$$

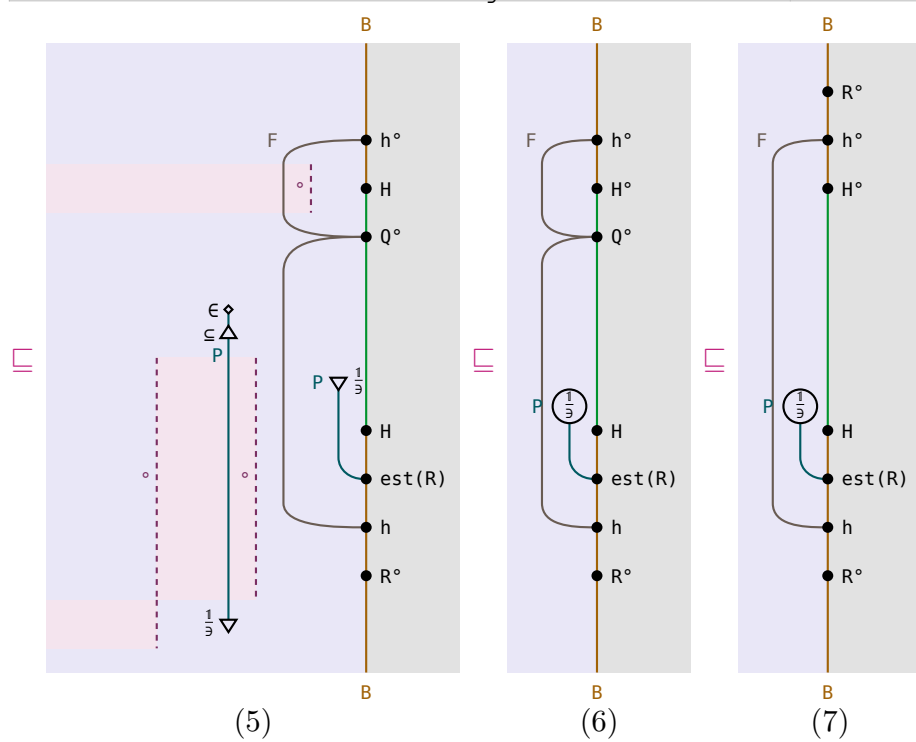
$$T^\circ F(H) h = H$$

$$\text{Map}(h) \wedge h \text{ Pres } R^\circ \wedge R^\circ R^\circ \sqsubseteq R^\circ \wedge \text{ThinCondition}(T, h, R, Q) \Rightarrow H^\circ \frac{T^\circ}{3} \text{thin}(Q) P(F(\frac{H}{3} \text{est}(R)) h) \text{est}(R) \sqsubseteq R^\circ \quad (9.1.1f)$$

$H^\circ$  followed by the body at  $M$  is  $\mathbb{E}R^\circ$ : an answer of the body is never worse than an answer of  $H$  to the same input



|   |                                                                                                                            |                                                                   |
|---|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| 1 | $H^\circ \frac{T^\circ}{3} \text{thin}(Q) P(F(\frac{H}{3} \text{est}(R)) h) \text{est}(R)$                                 | $P(S) \text{est}(R) \sqsubseteq \mathbb{E} \setminus (S R^\circ)$ |
| 2 | $\sqsubseteq H^\circ \frac{T^\circ}{3} \text{thin}(Q) (\mathbb{E} \setminus (F(\frac{H}{3} \text{est}(R)) h R^\circ))$     | $H^\circ = h^\circ F(H^\circ) T$                                  |
| 3 | $= h^\circ F(H^\circ) T \frac{T^\circ}{3} \text{thin}(Q) (\mathbb{E} \setminus (F(\frac{H}{3} \text{est}(R)) h R^\circ))$  | $T \frac{T^\circ}{3} \sqsubseteq \mathbb{E}$                      |
| 4 | $\sqsubseteq h^\circ F(H^\circ) \mathbb{E} \text{thin}(Q) (\mathbb{E} \setminus (F(\frac{H}{3} \text{est}(R)) h R^\circ))$ | $\mathbb{E} \text{thin}(Q) \sqsubseteq Q^\circ \mathbb{E}$        |



|   |                                                                                                                     |                                                                                 |
|---|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 5 | $\sqsubseteq h^\circ F(H^\circ) Q^\circ \mathbb{E} (\mathbb{E} \setminus (F(\frac{H}{3} \text{est}(R)) h R^\circ))$ | $S (S \setminus R) \sqsubseteq R$                                               |
| 6 | $\sqsubseteq h^\circ F(H^\circ) Q^\circ F(\frac{H}{3} \text{est}(R)) h R^\circ$                                     | $\text{ThinCondition}(T, h, R, Q)$                                              |
| 7 | $\sqsubseteq R^\circ h^\circ F(H^\circ) F(\frac{H}{3} \text{est}(R)) h R^\circ$                                     | $\Rightarrow h^\circ F(H^\circ) Q^\circ \sqsubseteq R^\circ h^\circ F(H^\circ)$ |

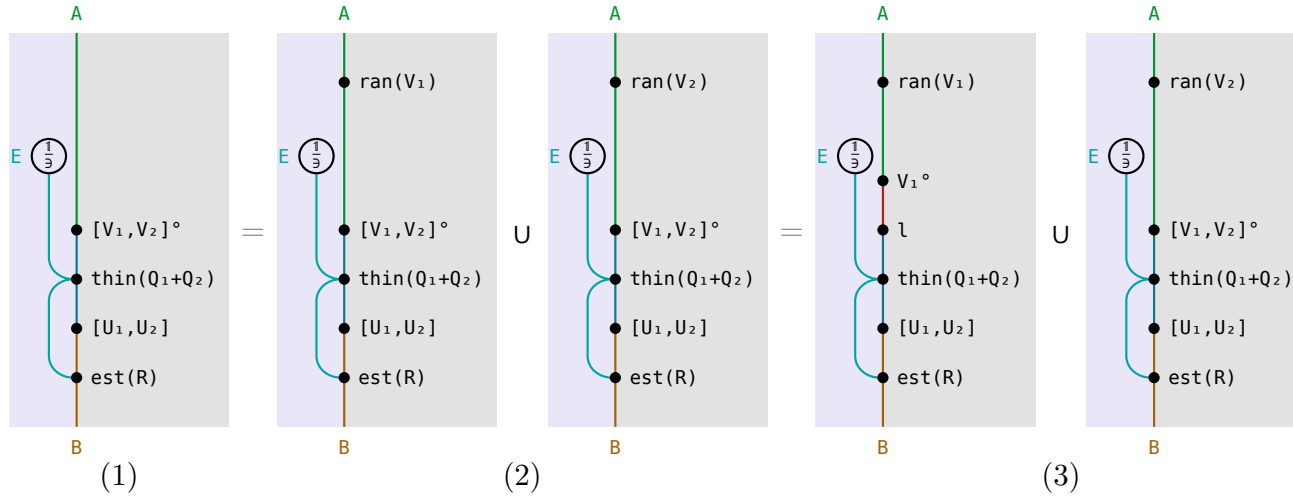
|    |                                                                                                              |                                                                                               |
|----|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|    |                                                                                                              |                                                                                               |
| 8  | $= R^\circ \quad h^\circ \quad F(H^\circ \quad \frac{H}{\exists} \quad \text{est}(R)) \quad h \quad R^\circ$ | $S^\circ \quad \frac{S}{\exists} \quad \text{est}(R) \quad \sqsubseteq R^\circ$               |
| 9  | $\sqsubseteq R^\circ \quad h^\circ \quad F(R^\circ) \quad h \quad R^\circ$                                   | $\text{Map}(f)$<br>$\implies f \text{ Pres } R \iff f^\circ F(R) \quad f \quad \sqsubseteq R$ |
| 10 | $\sqsubseteq R^\circ \quad R^\circ \quad R^\circ$                                                            | $R^\circ \quad R^\circ \quad \sqsubseteq R^\circ$                                             |
| 11 | $\sqsubseteq R^\circ \quad R^\circ$                                                                          | $R^\circ \quad R^\circ \quad \sqsubseteq R^\circ$                                             |
| 12 | $\sqsubseteq R^\circ$                                                                                        |                                                                                               |

### §9.1.2. Checking the conditions

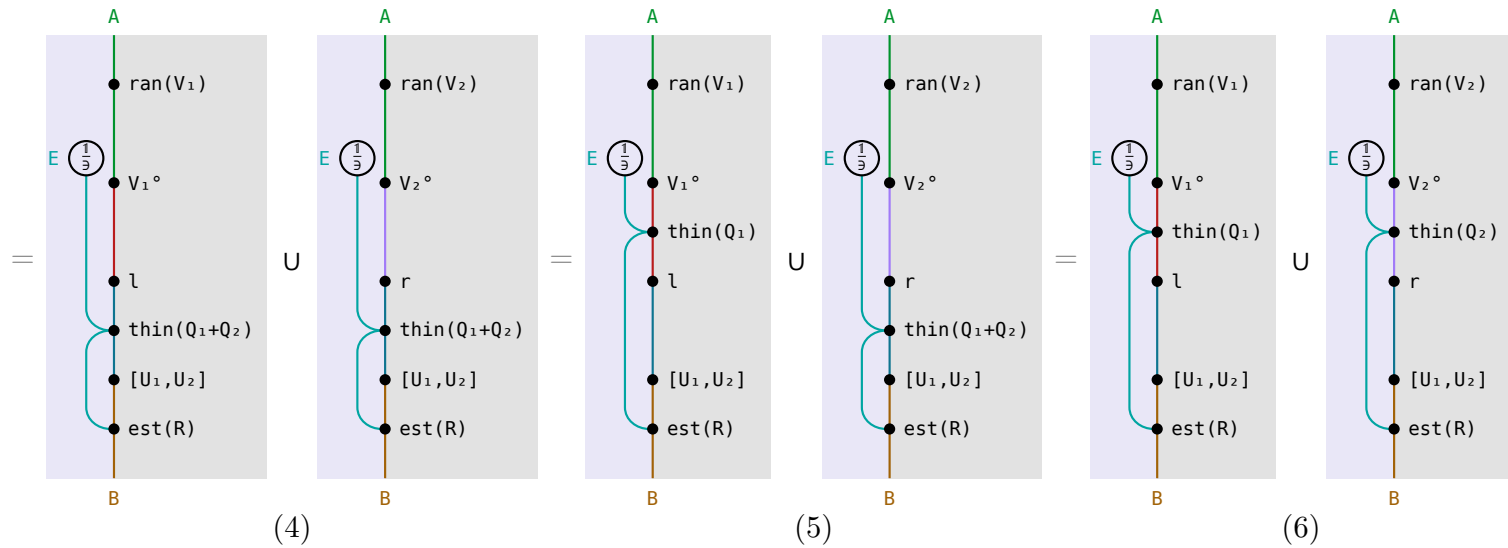
Proposition 9.1

$$V_2 \ V_1^\circ = \emptyset \implies \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R) = (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \text{thin}(Q_1) \ P(U_1) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \text{thin}(Q_2) \ P(U_2) \ \text{est}(R))$$

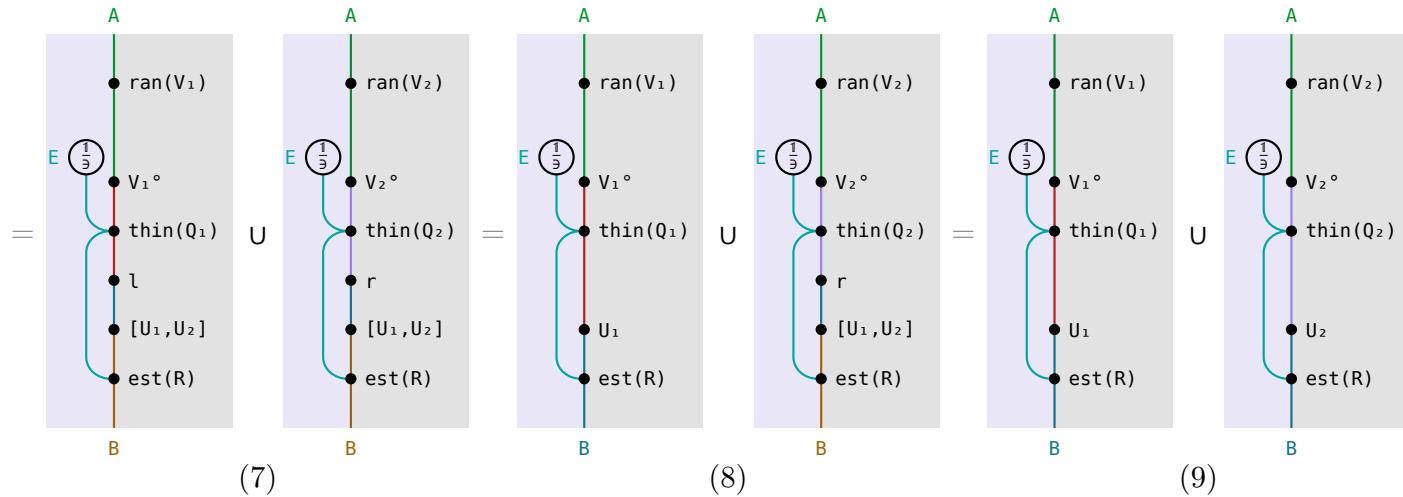
when  $V_1$  and  $V_2$  have disjoint ranges, the body over  $[V_1, V_2]$  runs the  $V_1$  problem on inputs  $V_1$  reaches and the  $V_2$  problem on inputs  $V_2$  reaches



|   |                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                        |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | $\frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)$                                                                                                                                    | $\frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R) = (\text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$ |
| 2 | $= (\text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$   | $V_2 \ V_1^\circ = \emptyset \implies \text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} = \text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l)$                                                                                                                                                                     |
| 3 | $= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$ | $V_2 \ V_1^\circ = \emptyset \implies \text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} = \text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r)$                                                                                                                                                                     |



|   |                                                                                                                                                                                                                          |                                                           |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| 4 | $= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$ | $P(l) \ \text{thin}(Q_1 + Q_2) = \text{thin}(Q_1) \ P(l)$ |
| 5 | $= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$       | $P(r) \ \text{thin}(Q_1 + Q_2) = \text{thin}(Q_2) \ P(r)$ |
| 6 | $= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ \text{est}(R))$             |                                                           |



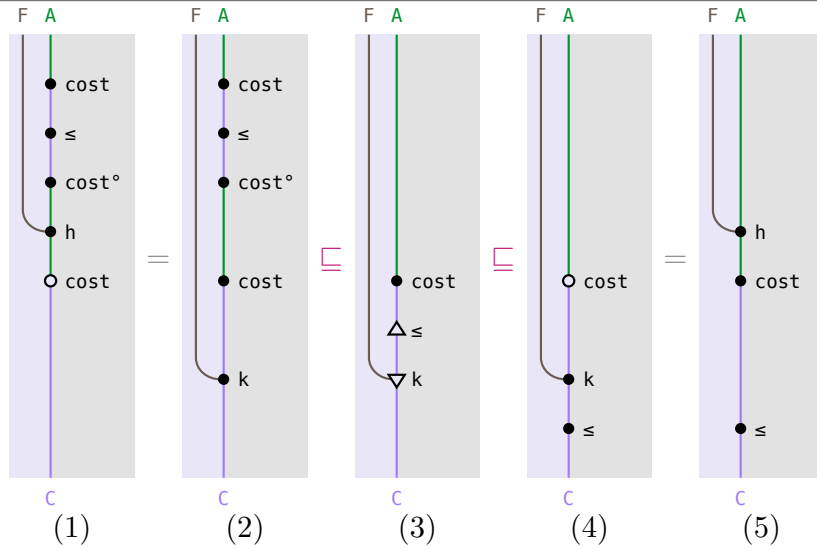
|   |                                                                                                                                                                                                              |                  |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| 7 | $= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ \text{est}(R))$ | $l \ [R, S] = R$ |
| 8 | $= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(U_1) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ \text{est}(R))$               | $r \ [R, S] = S$ |
| 9 | $= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(U_1) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(U_2) \ \text{est}(R))$                             |                  |

**Map(cost)  $\wedge$  h cost = F(cost) k  $\wedge$  F( $\leq$ ) k  $\sqsubseteq$  k  $\leq \implies$  F(cost  $\leq$  cost $^\circ$ ) h  $\sqsubseteq$  h cost  $\leq$  cost $^\circ$**  Proposition 9.2

if h then **cost** equals **F(cost)** then a **k** monotonic on  $\leq$ , then **h** is monotonic on **cost $\leq$ cost $^\circ$** ,  
which compares two values by comparing their **costs** under  $\leq$

**Map(cost)  $\wedge$  h cost = F(cost) k  $\wedge$  F( $\leq$ ) k  $\sqsubseteq$  k  $\leq \implies$  F(cost  $\leq$  cost $^\circ$ ) h cost  $\sqsubseteq$  h cost  $\leq$**

by shunting at the map **cost**, **F(cost $\leq$ cost $^\circ$ )h $\sqsubseteq$ h cost $\leq$ cost $^\circ$**  is **F(cost $\leq$ cost $^\circ$ )h cost $\sqsubseteq$ h cost  $\leq$** , which the chain proves



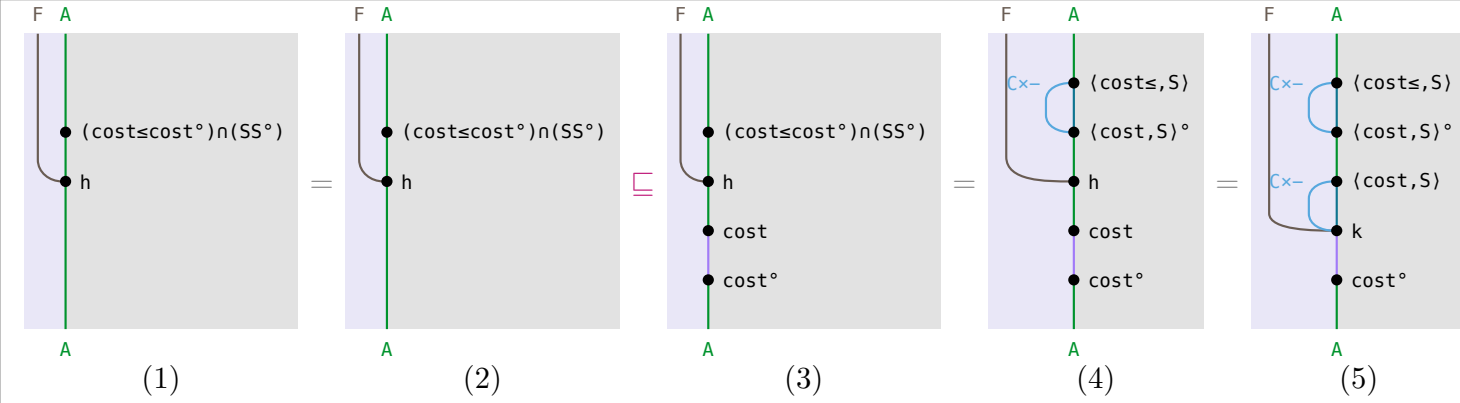
|   |                                                                    |                                               |
|---|--------------------------------------------------------------------|-----------------------------------------------|
| 1 | $F(\text{cost} \leq \text{cost}^\circ) \text{ h cost}$             | $\text{h cost} = F(\text{cost}) \text{ k}$    |
| 2 | $= F(\text{cost} \leq \text{cost}^\circ) F(\text{cost}) \text{ k}$ | $\text{Map}(f) \wedge R = f S f^\circ$        |
| 3 | $\sqsubseteq F(\text{cost}) F(\leq) \text{ k}$                     | $\implies R f \sqsubseteq f S$                |
| 4 | $\sqsubseteq F(\text{cost}) \text{ k} \leq$                        | $F(\leq) \text{ k} \sqsubseteq \text{k} \leq$ |
| 5 | $= \text{h cost} \leq$                                             | $\text{h cost} = F(\text{cost}) \text{ k}$    |

(9.1.2a)

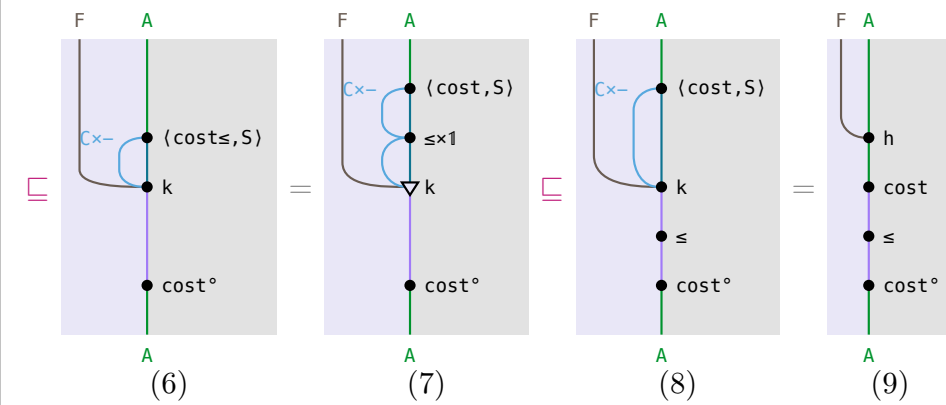
|   | the condition                                                                                                                                                                                                                                                                              | how it is discharged                                                                      |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 1 | $F(R) \text{ h} \sqsubseteq \text{h} R$<br>Proposition 9.2, $R \triangleq \text{cost} \leq \text{cost}^\circ$ , $\text{h cost} = F(\text{cost}) \text{ k}$ , $F(\leq) \text{ k} \sqsubseteq \text{k} \leq$                                                                                 | monotonicity when the cost is itself a fold with a step <b>k</b> monotonic on $\leq$      |
| 2 | $F(R \text{ n } (H^\circ H)) \text{ h} \sqsubseteq \text{h} R$<br>Proposition 9.3, $R \triangleq \text{cost} \leq \text{cost}^\circ$ , $\text{h cost} = F(\langle \text{cost}, H^\circ \rangle) \text{ k}$ ,<br>$F(\leq \times 1) \text{ k} \sqsubseteq \text{k} \leq$ , $H^\circ$ simple; | monotonicity <b>in context</b> : <b>k</b> may also read the input the part was built from |
| 3 | $QF(H) \text{ h} \sqsubseteq F(H) \text{ h} R$ at $Q \triangleq F(U, V)$<br>Proposition 9.4, $U, V$ preorders, $F(U, R) \text{ h} \sqsubseteq \text{h} R$ , $V H \sqsubseteq H R$                                                                                                          | both conditions at once, split along the two arguments of a bifunctor                     |

**Map(cost)  $\wedge$  Simple(S)  $\wedge$  h cost = F((cost,S)) k  $\wedge$  F( $\leq \times 1$ ) k  $\sqsubseteq$  k  $\leq \Rightarrow$  F((cost  $\leq$  cost $^\circ$ )  $\cap$  (S S $^\circ$ )) h  $\sqsubseteq$  h cost  $\leq$  cost $^\circ$**  Proposition 9.3

improving each part by **R** within its **S**-context, then assembling by **h**, is below **hR**



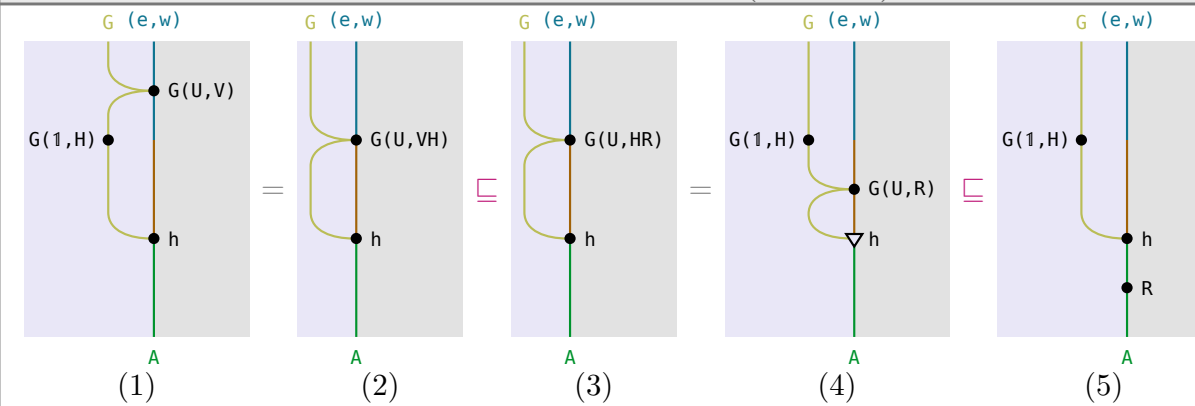
|   |                                                                                                  |                                                                    |
|---|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| 1 | $F((\text{cost} \leq \text{cost}^\circ) \cap (S S^\circ)) h$                                     |                                                                    |
| 2 | $= F((\text{cost} \leq \text{cost}^\circ) \cap (S S^\circ)) h$                                   | Map(f)                                                             |
| 3 | $\sqsubseteq F((\text{cost} \leq \text{cost}^\circ) \cap (S S^\circ)) h \text{ cost cost}^\circ$ | $\Rightarrow 1 \sqsubseteq f f^\circ$                              |
| 4 | $= F((\text{cost} \leq S) (\text{cost}, S)^\circ) h \text{ cost cost}^\circ$                     | $\langle X, Y \rangle (R, S)^\circ = (X R^\circ) \cap (Y S^\circ)$ |
| 5 | $= F((\text{cost} \leq S) (\text{cost}, S)^\circ (\text{cost}, S)) k \text{ cost}^\circ$         | $h \text{ cost} = F((\text{cost}, S)) k$                           |



|   |                                                               |                                                              |
|---|---------------------------------------------------------------|--------------------------------------------------------------|
| 6 | $\sqsubseteq F((\text{cost} \leq S)) k \text{ cost}^\circ$    | $\langle X, Y \rangle (R \times 1) = \langle X R, Y \rangle$ |
| 7 | $= F((\text{cost}, S)) F(\leq \times 1) k \text{ cost}^\circ$ | $F(\leq \times 1) k \sqsubseteq k \leq$                      |
| 8 | $\sqsubseteq F((\text{cost}, S)) k \leq \text{cost}^\circ$    | $h \text{ cost} = F((\text{cost}, S)) k$                     |
| 9 | $= h \text{ cost} \leq \text{cost}^\circ$                     |                                                              |

**G(U,R) h  $\sqsubseteq$  h R  $\wedge$  V H  $\sqsubseteq$  H R  $\Rightarrow$  G(U,V) G(1,H) h  $\sqsubseteq$  G(1,H) h R** Proposition 9.4

thinning the parts by **U** in the first argument and by **V** in the second (**G(U,V)**), then solving by **H** and assembling by **h**, gives only what solving and assembling and then improving by **R** gives (**G(1,H)hR**)



|   |                           |                            |
|---|---------------------------|----------------------------|
| 1 | $G(U,V) G(1,H) h$         |                            |
| 2 | $= G(U,V H) h$            | $V H \sqsubseteq H R$      |
| 3 | $\sqsubseteq G(U, H R) h$ |                            |
| 4 | $= G(1,H) G(U,R) h$       | $G(U,R) h \sqsubseteq h R$ |
| 5 | $\sqsubseteq G(1,H) h R$  |                            |

## §9.2. The string edit problem

(9.2a)

|    | name        | type                                                                            | definition                                                                                                                                                                                                                                                                                                                                                                                                     | meaning                                                                                                                     |
|----|-------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 1  | Op          | Type                                                                            | $(\exists a. o = \text{cpy}(a)) \vee (\exists a. o = \text{del}(a)) \vee (\exists a. o = \text{ins}(a))$                                                                                                                                                                                                                                                                                                       | an edit operation copies, deletes or inserts one character                                                                  |
| 2  |             |                                                                                 | $FX = 1 + (Op \times X)$                                                                                                                                                                                                                                                                                                                                                                                       | $F(Op, X)$ is the empty sequence or one operation paired with an $X$                                                        |
| 3  | edit        | $[Op] \rightarrow [Char] \times [Char]$                                         | $\text{edit} = ([\text{base}, \text{step}])$ , $\text{edit}([]) = ([], [])$ ,<br>$\text{edit}([op] \# es) = \text{step}(r((op, \text{edit}(es))))$                                                                                                                                                                                                                                                             | the two strings an edit sequence turns one into the other                                                                   |
| 4  |             |                                                                                 | $\text{length} = ([\text{zero}, \pi_2 \text{ succ}])$                                                                                                                                                                                                                                                                                                                                                          | the number of operations in a sequence                                                                                      |
| 5  | R           | $[Op] \rightarrow [Op]$                                                         | $R = \text{length} \leq \text{length}^\circ$ , $R(es, fs) \iff \text{length}(es) \leq \text{length}(fs)$                                                                                                                                                                                                                                                                                                       | $es$ is no longer than $fs$                                                                                                 |
| 6  | V           | $[Char] \times [Char] \rightarrow [Char] \times [Char]$                         | $V = \text{suffix}^\circ \times \text{suffix}^\circ$ , $V(p, q) \iff \text{suffix}(\pi_1(p), \pi_1(q)) \wedge \text{suffix}(\pi_2(p), \pi_2(q))$                                                                                                                                                                                                                                                               | each string of the first pair is a suffix of the matching string of the second                                              |
| 7  | Q           | $F([Char] \times [Char]) \rightarrow F([Char] \times [Char])$                   | $l(val) \ Q \ l(val') \iff \text{true}$ , $r(p) \ Q \ r(q) \iff V(\pi_2(p), \pi_2(q))$ , $(\forall val. \forall val'. u = l(val) \implies v = l(val') \implies \text{false}) \wedge (\forall p. \forall q. u = r(p) \implies v = r(q) \implies \text{false}) \implies (u \ Q \ v \iff \text{false})$ ,<br>$Q(l(u), l(u))$ , $Q(r(p), r(q)) \iff V(\pi_2(p), \pi_2(q))$                                         | the identity on the <b>base</b> summand; on the <b>step</b> summand any two operations, and the two pairs of strings by $V$ |
| 8  | empty       | $[Char] \times [Char] \rightarrow [Char] \times [Char]$                         | $\text{empty}(p, q) \iff p = q \wedge p = ([], [])$                                                                                                                                                                                                                                                                                                                                                            | holds only at the pair of two empty strings                                                                                 |
| 9  | unstep      | $[Char] \times [Char] \rightarrow [Op \times [Char] \times [Char]]$             | $q \in \text{unstep}(p) \implies \text{step}(r(q)) = p$ , $\text{unstep}([[], []]) = []$ , $\text{unstep}([a] \# xs, []) = [(\text{del}(a), (xs, []))]$ ,<br>$\text{unstep}([[], b] \# ys) = [(\text{ins}(b), ([], ys))]$ ,<br>$\text{unstep}([a] \# xs, [b] \# ys) = \text{if } a = b \text{ then } [(\text{cpy } a, xs, ys)] \text{ else } [(\text{del } a, xs, [b] \# ys), (\text{ins } b, [a] \# xs, ys)]$ | every split of a pair of strings into one operation and a smaller pair; <b>step</b> puts it back                            |
| 10 | [nil, cons] | $L + E \times \mu F \rightarrow \mu F$                                          | $[\text{nil}, \text{cons}](l(u)) = []$ , $[\text{nil}, \text{cons}](r((o, es))) = [o] \# es$                                                                                                                                                                                                                                                                                                                   | $\alpha = [\text{nil}, \text{cons}]$ : the empty edit sequence, or one operation in front of a sequence                     |
| 11 | step        | $\text{Unit} + Op \times [Char] \times [Char] \rightarrow [Char] \times [Char]$ | $\text{step}(l(u)) = ([], [])$ , $\text{step}(r((\text{cpy}(a), (xs, ys)))) = ([a] \# xs, [a] \# ys)$ , $\text{step}(r((\text{del}(a), (xs, ys)))) = ([a] \# xs, ys)$ , $\text{step}(r((\text{ins}(a), (xs, ys)))) = (xs, [a] \# ys)$                                                                                                                                                                          | $[\text{base}, \text{step}]$ : the two empty strings, or one operation added to the front of the pair                       |

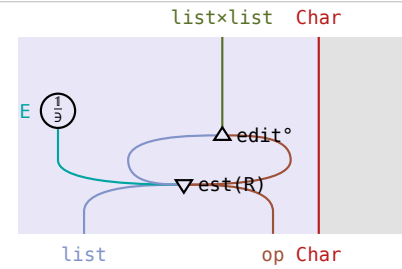
$$\text{mle} = (\text{empty} \rightarrow \text{nil}, \text{unstep list}((1 \times \text{mle}) \text{cons}) \text{minlist}(R)), (\mu X : \frac{[\text{base}, \text{step}]^\circ}{3} \text{thin}(Q) \text{P}(\text{F}(X) [\text{nil}, \text{cons}]) \text{est}(R)) \sqsubseteq \frac{\text{edit}^\circ}{3} \text{est}(R) \quad (9.2b)$$

a shortest edit sequence from which both strings can be reconstituted is one pass over the two of them, each step copying, deleting or inserting one character and the best sequence for what is left taken from the entries already computed

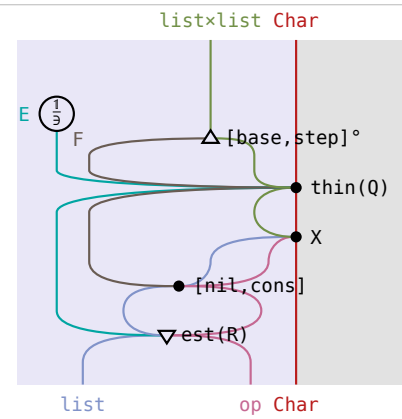
## circuit

$[\text{Char}] \xrightarrow{\frac{1}{3}} \text{P}[\text{Char}]^2 \xrightarrow{\text{E}(\text{edit}^\circ)} \text{P}[0p] \xrightarrow{\text{est}(R)} [0p]$   
 the specification —  $\text{edit}([\text{ }]) = ([\text{ }], [\text{ }])$ ,  $\text{edit}([op] \# es) = \text{step}(r((op, \text{edit}(es))))$

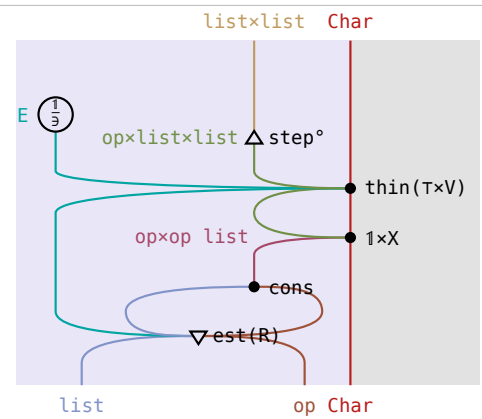
## Hinze–Marsden



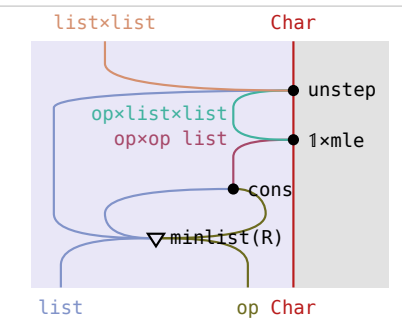
$[\text{Char}] \xrightarrow{\frac{1}{3}} \text{P}[\text{Char}]^2 \xrightarrow{\text{E}([\text{base}, \text{step}]^\circ)} \text{thin}(Q) \text{P}[\text{Char}]^2 \xrightarrow{\text{P}(\text{F}(X) [\text{nil}, \text{cons}])} \text{P}[0p] \xrightarrow{\text{est}(R)} [0p]$   
 at  $Q \triangleq 1 + (U \times V)$ . Monotonicity  $F(R) \alpha \sqsubseteq \alpha R$  is Proposition 9.2 at  $\text{length} \triangleq ([\text{zero}, \pi_2 \text{succ}])$  with  $\text{succ}$  monotonic on  $\leq$ , so  $\text{cons}$  is monotonic on  $R$ . The thinning condition  $\sqsubseteq QF(1, \text{edit}^\circ) \alpha \sqsubseteq F(1, \text{edit}^\circ) \alpha R$  over  $F(0p, [\text{Char}] \times [\text{Char}])$  is Proposition 9.4 at  $U \triangleq T$  —  $F(T, R) \alpha \sqsubseteq \alpha R$ , left as an exercise, so any two operations may be compared — and  $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$ , —  $\text{edit}(1 \times \text{suffix}) \sqsubseteq R^\circ \text{edit}$ ,  $\text{edit}(\text{suffix} \times 1) \sqsubseteq R^\circ \text{edit}$ .  $\text{suffix} = \text{tail}^*$  and  $BA \sqsubseteq CB \Rightarrow BA^* \sqsubseteq C^*B$  cut those to one step each: drop the operation that produced the head, or weaken its  $\text{cpy}$  to a  $\text{del}$ , never lengthening the sequence



$[\text{Char}] \xrightarrow{\frac{1}{3}} \text{P}[\text{Char}]^2 \xrightarrow{\text{E}(\text{step}^\circ)} \text{thin}(T \times V) \text{P}(0p \times [\text{Char}] \times [\text{Char}]) \xrightarrow{\text{P}((1 \times X) \text{cons})} \text{P}[0p] \xrightarrow{\text{est}(R)} [0p]$   
 = Proposition 9.1:  $\text{base}$  and  $\text{step}$  have disjoint ranges, so the fixed point splits into one branch per summand —  $\text{empty}$ , the coreflexive on  $(xs, ys)$  with both lists empty, is where  $\text{base}$  returns



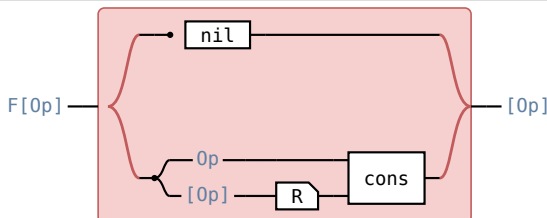
$[\text{Char}] \xrightarrow{\text{unstep}} [0p \times [\text{Char}] \times [\text{Char}]] \xrightarrow{\text{list}((1 \times \text{mle}) \text{cons})} [[0p]] \xrightarrow{\text{minlist}(R)} [0p]$   
 $\text{unstep}$  implements  $\frac{\text{step}^\circ}{3} \text{thin}(U \times V)$  — at most two decompositions survive, a copy beating a delete or an insert wherever it is available — and  $\text{minlist}(R)$  implements  $\text{est}(R)$ . The same subproblem is solved many times over, so the running time is exponential in the two lengths



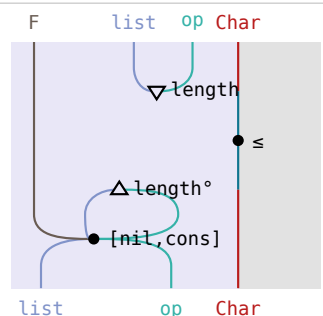
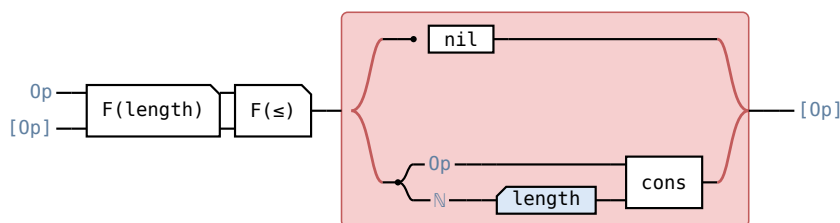
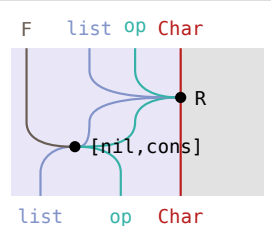
$$F(R) [\text{nil}, \text{cons}] \sqsubseteq [\text{nil}, \text{cons}] R \quad (9.2c)$$

comparing the tails by  $R$  and then putting an operation in front ( $F(R) \alpha$ ) relates only sequences that  $R$  also relates once the operation is in front ( $\alpha R$ )

## circuit



## Hinze–Marsden

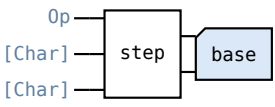
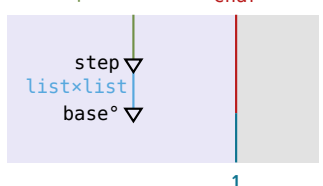
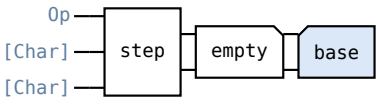
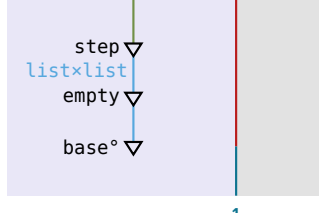
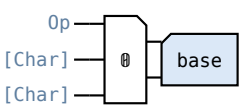
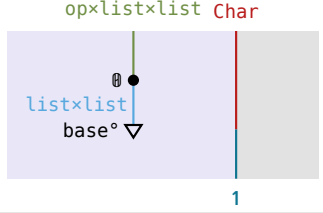
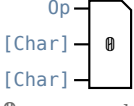
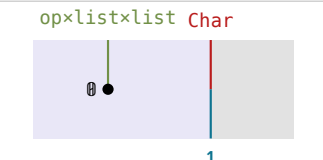


$R \triangleq \text{length} \leq \text{length}^\circ$  —  $es \ R \ fs \iff \text{length}(es) \leq \text{length}(fs)$ ;  $F$  preserves composition

| circuit                                                                                                                                                                                                                                            | Hinze–Marsden |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <p><math>\text{length}</math> is a map, so entire: <math>1 \sqsubseteq \text{length} \text{ length}^\circ</math></p>                                                                                                                               |               |
| <p><math>\alpha \text{ length} = F(\text{length})[\text{zero}, \pi_2 \text{ succ}] : \text{length} \triangleq ([\text{zero}, \pi_2 \text{ succ}]) - \text{es } R \text{ fs} \iff \text{length}(\text{es}) \leq \text{length}(\text{fs})</math></p> |               |
| <p><math>\text{length}</math> is a map, so simple, and <math>F</math> is monotonic: <math>F(\text{length}^\circ)F(\text{length}) \sqsubseteq F(1) = 1</math></p>                                                                                   |               |
| <p><math>\text{succ}</math> is monotonic on <math>\leq</math>: <math>F(\leq)[\text{zero}, \pi_2 \text{ succ}] \sqsubseteq [\text{zero}, \pi_2 \text{ succ}] \leq</math></p>                                                                        |               |
| <p><math>F(\text{length})[\text{zero}, \pi_2 \text{ succ}] = \alpha \text{ length}</math> again</p>                                                                                                                                                |               |
| <p><math>R \triangleq \text{length} \leq \text{length}^\circ - \text{es } R \text{ fs} \iff \text{length}(\text{es}) \leq \text{length}(\text{fs})</math></p>                                                                                      |               |

| $Q \vdash F(\text{edit}^\circ) [\text{nil}, \text{cons}] \sqsubseteq F(\text{edit}^\circ) [\text{nil}, \text{cons}] R$ (9.2d)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| every edit sequence built from a decomposition $Q$ puts above a given one — any operation, each string lengthened at the front — is at least as long as one built from the given one                                                                                                                                                                                                                                                                                                                                                                                                                                                     |               |
| circuit                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Hinze–Marsden |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |               |
| $=$ <p> <math>Q \triangleq 1 + (U \times V)</math> is <math>F(U, V)</math> at <math>U \triangleq \tau - l(\text{val}) \ Q \ l(\text{val}') \iff \text{true}</math>, <math>r(p) \ Q \ r(q) \iff V(\pi_2(p), \pi_2(q))</math>, <math>(\forall \text{val}. \forall \text{val}'. u = l(\text{val}) \implies v = l(\text{val}') \implies \text{false}) \wedge (\forall p. \forall q. u = r(p) \implies v = r(q) \implies \text{false})</math><br/> <math>\implies (u \ Q \ v \iff \text{false})</math>; and the bifunctor <math>F</math> preserves composition: <math>F(U, V)F(1, \text{edit}^\circ) = F(U, V \text{ edit}^\circ)</math> </p> |               |
| $\sqsubseteq$ <p>Proposition 9.4's second condition <math>V \text{ edit}^\circ \sqsubseteq \text{edit}^\circ R - \text{edit} \ V^\circ \sqsubseteq R^\circ \text{ edit}</math></p>                                                                                                                                                                                                                                                                                                                                                                                                                                                       |               |
| $=$ <p><math>F</math> preserves composition: <math>F(U, \text{edit}^\circ R) = F(1, \text{edit}^\circ)F(U, R)</math></p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |               |
| $\sqsubseteq$ <p>Proposition 9.4's first condition <math>F(\tau, R) \alpha \sqsubseteq \alpha R</math>, left as an exercise in the book: <b>cons</b> adds one to both lengths whatever the two operations are</p>                                                                                                                                                                                                                                                                                                                                                                                                                        |               |

| $\text{edit } V^\circ \sqsubseteq R^\circ \text{ edit}$ (9.2e)<br>cutting a front off either string an edit sequence produces $(\text{edit } V^\circ)$ leaves a pair that a sequence no longer produces $(R^\circ \text{ edit})$                                                                                                                                                         |               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| circuit                                                                                                                                                                                                                                                                                                                                                                                  | Hinze–Marsden |
|                                                                                                                                                                                                                                                                                                                                                                                          |               |
| $=$ <p><math>V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ</math>, and <math>\times</math> preserves composition</p>                                                                                                                                                                                                                                                       |               |
| $\sqsubseteq$ <p><math>\text{edit } (\text{suffix} \times 1) \sqsubseteq R^\circ \text{ edit}</math>: drop the operation that produced the head, or weaken its <b>cpy</b> to an <b>ins</b>. Proved by induction at <b>suffix</b> directly, not through <math>\text{suffix} = \text{tail}^*</math> and <math>BA \sqsubseteq CB \implies BA^* \sqsubseteq C^*B</math> as the book does</p> |               |
| $\sqsubseteq$ <p><math>\text{edit } (1 \times \text{suffix}) \sqsubseteq R^\circ \text{ edit}</math>, the mirror image: <b>cpy</b> weakens to a <b>del</b></p>                                                                                                                                                                                                                           |               |
| $\sqsubseteq$ <p><math>R^\circ</math> is transitive, <math>\leq</math> being so</p>                                                                                                                                                                                                                                                                                                      |               |

| $\text{step } \text{base}^\circ = \emptyset$ (9.2f)<br>no pair of strings is both a result of <b>step</b> and the result $([], [])$ of <b>base</b>                                                                                     |                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| circuit                                                                                                                                                                                                                                | Hinze–Marsden                                                                        |
|                                                                                                                                                       |   |
|  <p><math>\text{base}</math> returns only <math>([], [])</math>, so <math>\text{base}^\circ = \text{empty } \text{base}^\circ</math></p>              |   |
|  <p><math>\text{step } \text{empty} = \emptyset</math>: <b>cpy</b> and <b>del</b> put a character on the left string, <b>ins</b> one on the right</p> |   |
|  <p><math>\emptyset</math> composed with anything is <math>\emptyset</math></p>                                                                      |  |

### §9.2.1. Tabulation

(9.2.1a)

|   |                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | $\text{mle}(xs, ys) = \text{head}(\text{column}(xs, ys))$ , $\text{column}(xs, ys) = [\text{mle}(u, ys) \mid u \leftarrow \text{tails}(xs)]$<br>$= \text{column}(xs) = ([\text{fstcol}(xs), \text{nextcol}(xs)] \triangleright)$ , $\text{fstcol} = \text{list}(\text{del}) \text{ tails}$<br>the tabulation: $\text{mle}(xs, ys)$ needs $\text{mle}(u, v)$ for every tail $u$ of $xs$ and $v$ of $ys$ , so the columns are built right to left           |
| 2 | $\text{column}(xs, [b] \# ys) = \text{nextcol}(xs, (b, \text{column}(xs, ys)))$<br>$= \text{nextcol}(xs)(b, us) = ([\text{base}(b, \text{last}(us)), \text{step}(b)] \triangleright)(xus)$ , $xus = \text{zip}(xs, \text{zip}(\text{init}(us), \text{tail}(us)))$<br>each column is a fold built bottom to top, over $xs$ zipped with the adjacent pairs of the column to its right                                                                       |
| 3 | $\text{base}(b, u) = [[\text{ins}(b)] \# u]$<br>$= \text{step}(b)((a, (u, v)), ws) = (a \rightarrow b \rightarrow [[\text{cpy}(a)] \# v] \# ws, [\text{bmin}(R)([\text{del}(a)] \# w, [\text{ins}(b)] \# u)] \# ws)$<br>$w = \text{head}(ws)$ ; these <b>base</b> , <b>step</b> are not <b>edit</b> 's. An entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths |

## §9.3. Optimal bracketing

(9.3a)

|    | name                                       | type                                                                    | definition                                                                                                                                                                                                                                           | meaning                                                                                                             |
|----|--------------------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| 1  | <code>tree(A)</code>                       | Type                                                                    | $\text{tree}(A) \triangleq \text{tip}(A) \mid \text{bin}(\text{tree}(A), \text{tree}(A))$                                                                                                                                                            | a tree is a tip holding one value, or a node joining two trees;<br>$\mathbf{h} \triangleq [\text{tip}, \text{bin}]$ |
| 2  | <code>flatten</code>                       | $\text{tree}(A) \rightarrow \text{LA}$                                  | $\langle [\text{wrap}, \text{cat}] \rangle = \text{flatten}, \text{flatten}(\text{tip}(a)) = [a],$<br>$\text{flatten}(\text{bin}(xs, ys)) = \text{cat}(\text{flatten}(xs), \text{flatten}(ys))$                                                      | the tips of a tree, left to right;<br>$\mathbf{H} = \text{flatten}^\circ$ builds every tree over a list             |
| 3  | <code>opb(p)</code>                        | $\text{Int} \times \text{S}$                                            | $\text{opb}(p) \triangleq ((\text{cb}(\pi_2(\pi_1(p)), \pi_2(\pi_2(p))) + \pi_1(\pi_1(p))) + \pi_1(\pi_2(p)), \text{sb}(\pi_2(\pi_1(p)), \pi_2(\pi_2(p))))$                                                                                          | joining two subtrees costs <b>cb</b> of their sizes plus the cost of each                                           |
| 4  | $\langle \text{cost}, \text{size} \rangle$ | $\text{tree}(A) \rightarrow \text{Int} \times \text{S}$                 | $\langle \text{cost}, \text{size} \rangle(\text{tip}(a)) = (0, \text{st}(a)), \langle \text{cost}, \text{size} \rangle(\text{bin}(xs, ys)) = \text{opb}(\langle \text{cost}, \text{size} \rangle(xs), \langle \text{cost}, \text{size} \rangle(ys))$ | the cost of evaluating a bracketing, paired with the size of its result                                             |
| 5  | <code>sz</code>                            | $\text{LA} \rightarrow \text{S}$                                        | $\text{sz}([a]) = \text{st}(a), \text{sz}([a] \# xs) = \text{sb}(\text{st}(a), \text{sz}(xs)),$<br>$\text{Assoc}(\text{sb}) \implies \pi_2(\langle \text{cost}, \text{size} \rangle(xs)) = \text{sz}(\text{flatten}(xs))$                            | <b>sb</b> associative, so the size depends only on the flattened list                                               |
| 6  | <code>R</code>                             | $\text{tree}(A) \rightarrow \text{tree}(A)$                             | $R = \text{cost} \leq \text{cost}^\circ, R(xs, ys) \iff \text{cost}(xs) \leq \text{cost}(ys)$                                                                                                                                                        | <b>t</b> costs no more than <b>t'</b>                                                                               |
| 7  | <code>g</code>                             | $\text{F}(\text{Int} \times \text{LA}) \rightarrow \text{Int}$          | $g \triangleq [\neg 0, (1 \times \text{sz})^2 \text{ opb } \pi_1], g(\text{l}(a)) = 0, g(\text{r}((cx, xs), cy, ys)) = (\text{cb}(\text{sz}(xs), \text{sz}(ys)) + cx) + cy$                                                                          | the cost of the top step alone: zero at a tip, <b>cb</b> of the two sizes at a node                                 |
| 8  | <code>nonsingle</code>                     | $\text{LA} \rightarrow \text{LA}$                                       | $xs \text{ nonsingle } ys \iff xs = ys \wedge (\exists a. \exists zs. xs = [a] \# zs)$                                                                                                                                                               | holds at the lists of two or more elements                                                                          |
| 9  | <code>splits</code>                        | $\text{LA} \rightarrow [\text{L } A \times \text{L } A]$                | $\text{splits} = (\text{inits}^+, \text{tails}^+) \text{ zip}, \text{splits}(xs) = \text{zip}(((\text{inits}^+)(xs)), ((\text{tails}^+)(xs)))$                                                                                                       | every way to cut a list into two non-empty parts — an implementation of $\frac{\text{cat}^\circ}{\exists}$          |
| 10 | <code>array</code>                         | $\text{LA} \rightarrow [\text{tree } A]$                                | $\text{array} \triangleq \text{inits list}(\text{row}), \text{array}(xs, xss) \iff xss = \text{list}(\text{row}, \text{inits}(xs))$                                                                                                                  | the best trees of every segment, held as rows                                                                       |
| 11 | <code>row</code>                           | $\text{LA} \rightarrow [\text{tree } A]$                                | $\text{row} = \text{tails list}(\text{mct}), \text{row}(xs, xss) \iff xss = \text{list}(\text{mct}, \text{tails}(xs))$                                                                                                                               | the best tree of every non-empty suffix                                                                             |
| 12 | <code>col</code>                           | $\text{LA} \rightarrow [\text{tree } A]$                                | $\text{col} = \text{inits list}(\text{mct}), \text{col}(xs, xss) \iff xss = \text{list}(\text{mct}, \text{inits}(xs))$                                                                                                                               | the best tree of every non-empty prefix                                                                             |
| 13 | <code>mix</code>                           | $[\text{tree } A] \times [\text{tree } A] \rightarrow \text{tree}(A)$   | $\text{mix} = \text{zip list}(\text{bin}) \text{ minlist}(R), \text{mix}(p, xs) \iff xs = \text{minlist}(R) (\text{list bin zip})$                                                                                                                   | join the matching prefix and suffix trees and keep a cheapest                                                       |
| 14 | <code>next</code>                          | $[\text{tree } A] \times [\text{tree } A] \rightarrow [\text{tree } A]$ | $\text{next} = (\pi_1, \text{mix}) \text{ snoc}, \text{next}(p, xss) \iff xss = \text{snoc}((\pi_1(p), \text{minlist}(R) (\text{list bin zip})))$                                                                                                    | add the next best tree to the end of the column                                                                     |

15

| name    | type                                                       | definition                                                                                                                                                                                      | meaning                                              |
|---------|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| process | $A \times [[\text{tree } A]] \rightarrow [\text{tree } A]$ | $\text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next}) ,$<br>$\text{process}((a, xsss), xss) \Leftrightarrow \text{loop}(\text{next})(([\text{tip } a], xsss), xss)$ | start the column at a tip and run next over the rows |

| $\text{Assoc}(\text{sb}) \Rightarrow F(\langle \text{cost}, \text{flatten} \rangle) \text{ g} = [\text{tip}, \text{bin}] \text{ cost}$ (9.5)<br>pairing each subtree with its cost and its flattening and then applying <b>g</b> gives what building the tree by <b>[tip, bin]</b> and taking its <b>cost</b> gives |               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| circuit                                                                                                                                                                                                                                                                                                             | Hinze–Marsden |
| $F(\text{tree}(A)) \rightarrow F(\langle \text{cost}, \text{flatten} \rangle) \rightarrow F(\text{Int} \times \text{LA}) \rightarrow \text{g} \rightarrow \text{Int}$                                                                                                                                               |               |
| $A + \text{tree}(A)^2 \rightarrow \text{Int}$ <p>definition of <b>g</b>; coproducts and products — <math>\text{g} \triangleq [\rightarrow 0, (1 \times \text{sz})^2 \text{ opb } \pi_1]</math></p>                                                                                                                  |               |
| $A + \text{tree}(A)^2 \rightarrow \text{Int}$ <p><math>\text{flatten sz} = \text{size}</math>, since <b>sb</b> is associative — <math>\text{sz}([a]) = \text{st}(a)</math>, <math>\text{sz}([a] \# xs) = \text{sb}(\text{st}(a), \text{sz}(xs))</math></p>                                                          |               |
| $A + \text{tree}(A)^2 \rightarrow \text{Int}$ <p><math>\langle \text{cost}, \text{size} \rangle \triangleq ([\text{opt}, \text{opb}])</math>, at a node <math>\text{bin}(\text{cost}, \text{size}) = (\text{cost}, \text{size})^2 \text{ opb}</math></p>                                                            |               |
| $A + \text{tree}(A)^2 \rightarrow \text{Int}$ <p><math>\text{tip cost} = \text{zero}</math>; products, <math>\langle \text{cost}, \text{size} \rangle \pi_1 = \text{cost}</math></p>                                                                                                                                |               |
| $A + \text{tree}(A)^2 \rightarrow \text{Int}$ <p>coproducts, <math>[\text{tip cost}, \text{bin cost}] = [\text{tip}, \text{bin}] \text{ cost}</math></p>                                                                                                                                                            |               |

| $F(\leq \times 1) \ g \sqsubseteq g \leq$ <span style="float: right;">(9.6)</span>                                                                                                                              |               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| raising the cost of either subtree ( $\leq \times 1$ in both slots) and then applying $g$ returns a value at least what $g$ returns on the costs before raising ( $g \leq$ )                                    |               |
| circuit                                                                                                                                                                                                         | Hinze–Marsden |
| $F(\text{Int} \times \text{LA}) \rightarrow \boxed{F(\leq \times 1)} \rightarrow \boxed{g} \rightarrow \text{Int}$                                                                                              |               |
| $A + \text{Int} \times \text{LA} \times \text{Int} \times \text{LA} \rightarrow$<br>$=$<br>definition of $g \rightarrow g \triangleq [\neg 0, (1 \times \text{sz})^2 \text{ opb } \pi_1]$                       |               |
| $A + \text{Int} \times \text{LA} \times \text{Int} \times \text{LA} \rightarrow$<br>$\sqsubseteq$<br>definition of $\text{opb}$ , and $+$ monotonic                                                             |               |
| $A + \text{Int} \times \text{LA} \times \text{Int} \times \text{LA} \rightarrow$<br>$\sqsubseteq$<br>$\leq$ reflexive, so $\text{zero} \sqsubseteq \text{zero} \leq$ ; coproducts                               |               |
| $F(\text{Int} \times \text{LA}) \rightarrow \boxed{g} \rightarrow \boxed{\leq} \rightarrow \text{Int}$<br>$=$<br>definition of $g \rightarrow g \triangleq [\neg 0, (1 \times \text{sz})^2 \text{ opb } \pi_1]$ |               |

$$\text{mct}=(\text{single}\rightarrow\text{head tip},\{\text{init col},\text{tail row}\}\text{mix}), \text{ Assoc}(\text{sb}) \Rightarrow (\mu X : \frac{[\text{wrap},\text{cat}]^\circ}{\mathfrak{g}} \text{P}([\text{tip},(X \times X) \text{bin}]) \text{est}(\text{R})) \sqsubseteq \frac{\text{flatten}^\circ}{\mathfrak{g}} \text{est}(\text{R}) \quad (9.3b)$$

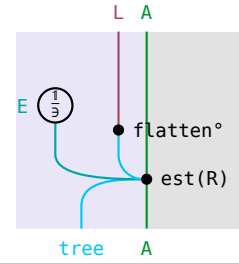
a least-cost bracketing of  $a_1 \oplus \dots \oplus a_n$  is read off an array holding one best tree per non-empty segment, each entry built from the column to its left and the row below it

## circuit

$$\text{LA} \rightarrow \frac{1}{\mathfrak{g}} \rightarrow \text{P}(\text{LA}) \rightarrow \text{E}(\text{flatten}^\circ) \rightarrow \text{P}(\text{tree}(\text{A})) \rightarrow \text{est}(\text{R}) \rightarrow \text{tree}(\text{A})$$

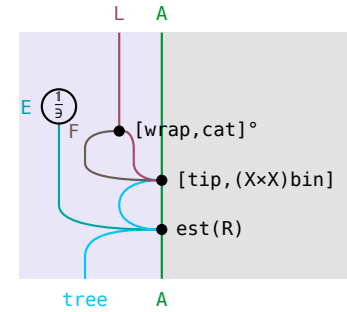
the specification —  $\text{flatten}(\text{tip}(\text{a})) = [\text{a}]$ ,  $\text{flatten}(\text{bin}(\text{xs},\text{ys})) = \text{cat}(\text{flatten}(\text{xs}),\text{flatten}(\text{ys}))$

## Hinze–Marsden



$$\text{LA} \rightarrow \frac{1}{\mathfrak{g}} \rightarrow \text{P}(\text{LA}) \rightarrow \text{E}([\text{wrap},\text{cat}]^\circ) \rightarrow \text{PF}(\text{LA}) \rightarrow \text{P}([\text{tip},(X \times X)\text{bin}]) \rightarrow \text{P}(\text{tree}(\text{A})) \rightarrow \text{est}(\text{R}) \rightarrow \text{tree}(\text{A})$$

split the list in every way, bracket both halves, join. The condition is monotonicity **in context**,  
 $\sqsubseteq \text{F}(\text{Rn}(\text{flatten} \text{ flatten}^\circ)) \text{h} \sqsubseteq \text{hR}$  — only trees with the same flattening are compared — which is Proposition 9.3 at  $\text{H}^\circ = \text{flatten}$ , a map, with (9.5)  $[\text{tip},\text{bin}] \text{ cost} = (1 + (\text{cost}, \text{flatten})^2) \text{g}$  (the cost of a node reads only the cost and the flattening of its two subtrees) and (9.6)  $(1 + (\leq \times 1)^2) \text{g} \sqsubseteq \text{g} \leq (\text{g} \text{ monotonic on } \leq \text{ in its two cost arguments})$

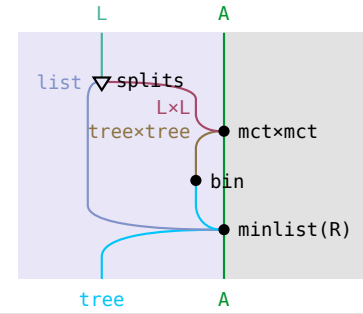


$$\text{LA} \rightarrow \frac{1}{\mathfrak{g}} \rightarrow \text{P}(\text{LA}) \rightarrow \text{E}(\text{cat}^\circ) \rightarrow \text{P}(\text{LA}^2) \rightarrow \text{P}((X \times X)\text{bin}) \rightarrow \text{P}(\text{tree}(\text{A})) \rightarrow \text{est}(\text{R}) \rightarrow \text{tree}(\text{A})$$

= Proposition 9.1: **wrap** and **cat** have disjoint ranges, and **single** is the coreflexive on singleton lists, where **wrap** returns

$$\text{LA} \rightarrow \text{splits} \rightarrow [\text{LA}^2] \rightarrow \text{list}((\text{mct} \times \text{mct})\text{bin}) \rightarrow [\text{tree}(\text{A})] \rightarrow \text{minlist}(\text{R}) \rightarrow \text{tree}(\text{A})$$

$\sqsubseteq \text{splits} \triangleq \langle \text{inits}^+, \text{tails}^+ \rangle$  zip implements  $\frac{\text{cat}^\circ}{\mathfrak{g}}$  and  $\text{minlist}(\text{R})$  implements  $\text{est}(\text{R})$ . Exponential, since the segments of one list overlap



$$\text{mct}=(\text{single}\rightarrow\text{head tip},\{\text{init col},\text{tail row}\}\text{mix}) \quad (9.7)$$

= the tabulation:  $\text{mct } \text{xs}$  is needed for every non-empty segment  $\text{xs}$ , so the values are held as an array of rows,  $\text{array} \triangleq \text{inits list}(\text{row}), \text{row} \triangleq \text{tails list}(\text{mct}), \text{col} \triangleq \text{inits list}(\text{mct}), \text{mix} \triangleq \text{zip list}(\text{bin}) \text{ minlist}(\text{R})$

$$\text{col}=(\text{single}\rightarrow\text{head tip wrap},\{\text{init col},\text{tail row}\}\text{next}) \quad (9.8)$$

$$\text{cons col}=(1 \times \text{array}) \text{ process} \quad (9.9)$$

$$\text{row}=(\text{single}\rightarrow\text{head tip wrap},\{\text{mct},\text{tail row}\}\text{cons}) \quad (9.10)$$

a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop,  $\text{next} \triangleq \langle \pi_1, \text{mix} \rangle \text{ snoc}, \text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next})$

$$\text{array}=(\text{fstcol},\text{addcol}), \text{fstcol} \triangleq \text{tip wrap wrap}$$

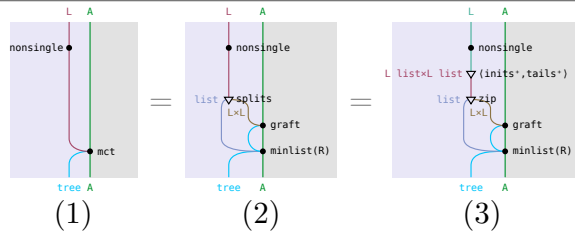
$$\text{addcol} \triangleq \langle \pi_1 \text{ tip wrap}, \text{step} \rangle \text{ cons}, \text{step} \triangleq (\text{process tail}, \pi_2) \text{ zip list}(\text{cons})$$

the program: one fold building the array column by column, cubic in the length of the input

**nonsingle mct = nonsingle (init col,tail row) mix**

(9.7)

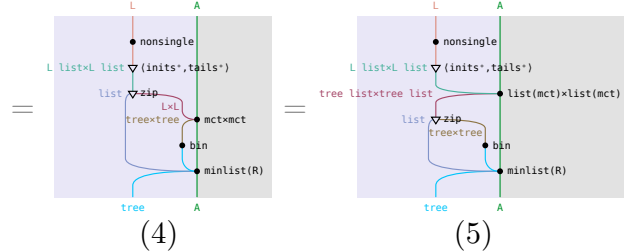
on a list of two or more elements, **mct** gives what **mix** gives on the column of its **init** and the row of its **tail**



(2)

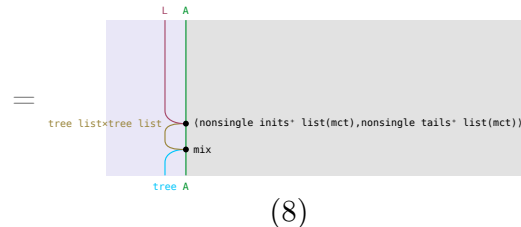
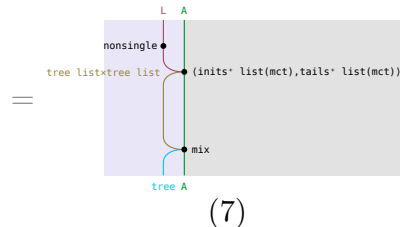
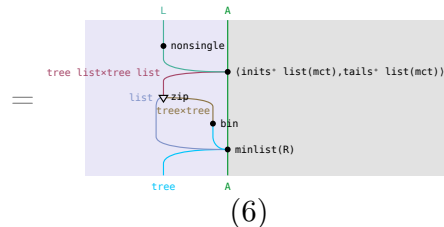
(3)

|   |                                                                                   |                                                         |
|---|-----------------------------------------------------------------------------------|---------------------------------------------------------|
| 1 | nonsingle mct                                                                     | nonsingle splits list(graft)<br>minlist(R)              |
| 2 | = nonsingle splits list(graft) minlist(R)                                         | splits = (inits <sup>+</sup> , tails <sup>+</sup> ) zip |
| 3 | = nonsingle (inits <sup>+</sup> , tails <sup>+</sup> ) zip list(graft) minlist(R) | graft $\triangleq$ (mct $\times$ mct) bin               |

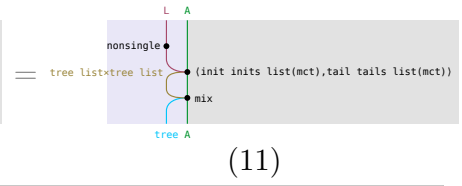
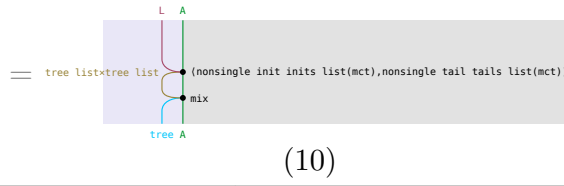
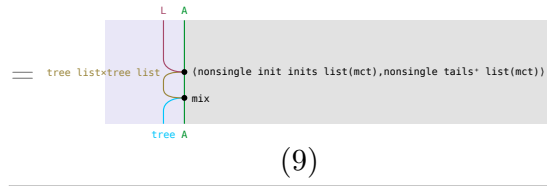


(5)

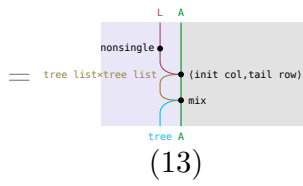
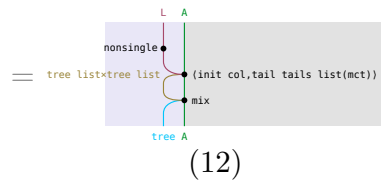
|   |                                                                                                                |                                                             |
|---|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| 4 | = nonsingle (inits <sup>+</sup> , tails <sup>+</sup> ) zip list(mct $\times$ mct) list(bin) minlist(R)         | zip list(f $\times$ g) M = (list(f) $\times$ list(g)) zip M |
| 5 | = nonsingle (inits <sup>+</sup> , tails <sup>+</sup> ) (list(mct) $\times$ list(mct)) zip list(bin) minlist(R) | (R, S) (U $\times$ V) = (R U, S V)                          |



|   |                                                                                                   |                                                         |
|---|---------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| 6 | = nonsingle (inits <sup>+</sup> list(mct), tails <sup>+</sup> list(mct)) zip list(bin) minlist(R) | mix = zip list(bin) minlist(R)                          |
| 7 | = nonsingle (inits <sup>+</sup> list(mct), tails <sup>+</sup> list(mct)) mix                      | nonsingle (U, V) = (nonsingle U, nonsingle V)           |
| 8 | = (nonsingle inits <sup>+</sup> list(mct), nonsingle tails <sup>+</sup> list(mct)) mix            | nonsingle inits <sup>+</sup> M = nonsingle init inits M |



|    |                                                                                |                                                         |
|----|--------------------------------------------------------------------------------|---------------------------------------------------------|
| 9  | = (nonsingle init inits list(mct), nonsingle tails <sup>+</sup> list(mct)) mix | nonsingle tails <sup>+</sup> M = nonsingle tail tails M |
| 10 | = (nonsingle init inits list(mct), nonsingle tail tails list(mct)) mix         | nonsingle (U, V) = (nonsingle U, nonsingle V)           |
| 11 | = nonsingle (init inits list(mct), tail tails list(mct)) mix                   | col = inits list(mct)                                   |

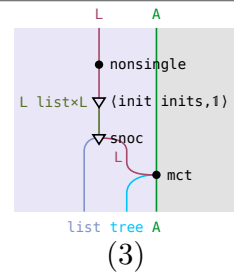
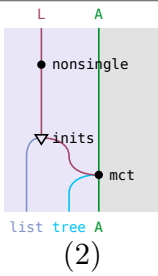
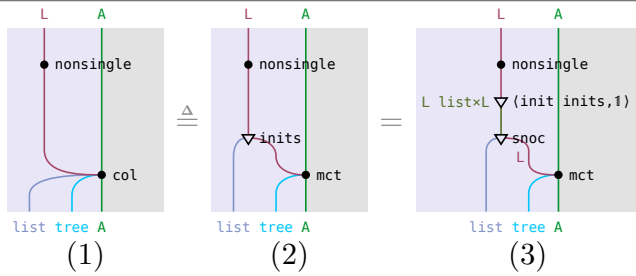


|    |                                                  |                       |
|----|--------------------------------------------------|-----------------------|
| 12 | = nonsingle (init col, tail tails list(mct)) mix | row = tails list(mct) |
| 13 | = nonsingle (init col, tail row) mix             |                       |

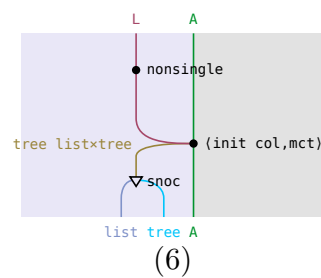
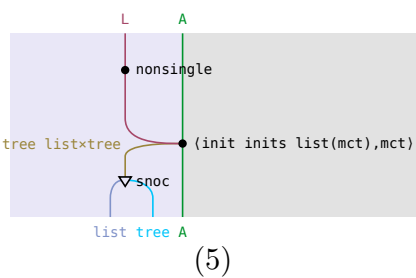
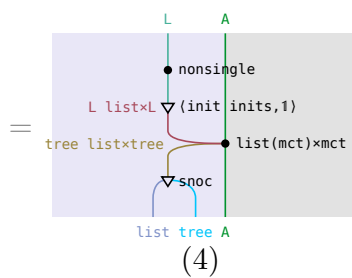
**nonsingle col = nonsingle (init col,tail row) next**

(9.8)

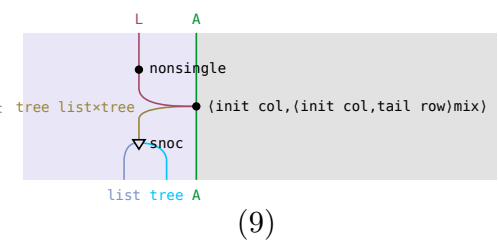
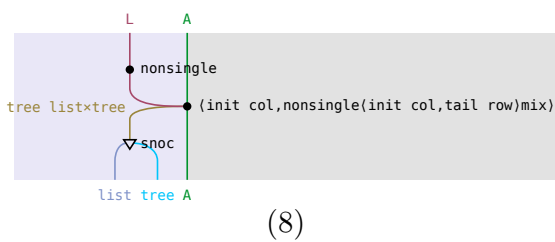
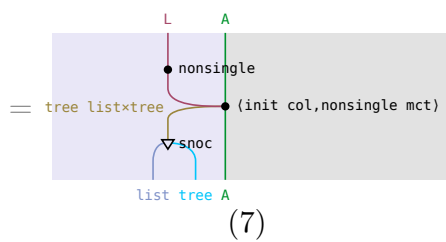
on a list of two or more elements, the column is what **next** makes of the column of its **init** and the row of its **tail**



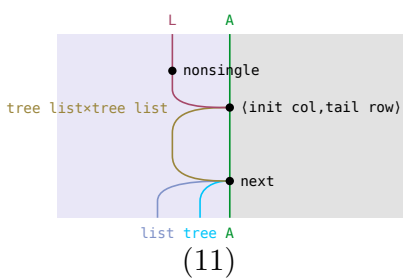
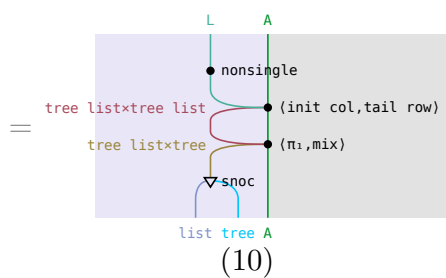
|   |                                            |                                                         |
|---|--------------------------------------------|---------------------------------------------------------|
| 1 | nonsingle col                              | col $\triangleq$ inits list(mct)                        |
| 2 | $\triangleq$ nonsingle inits list(mct)     | nonsingle inits M = nonsingle (init inits, 1)<br>snoc M |
| 3 | = nonsingle (init inits, 1) snoc list(mct) | snoc list(f) = (list(f) $\times$ f) snoc                |



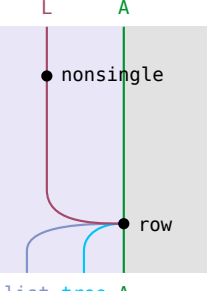
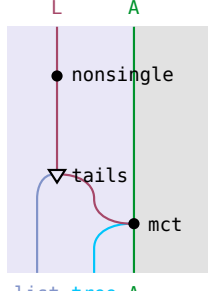
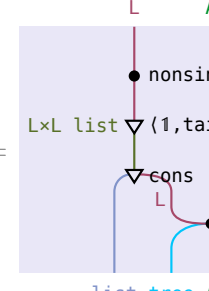
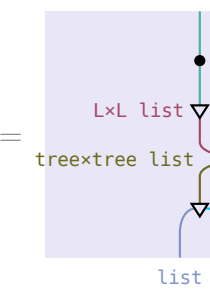
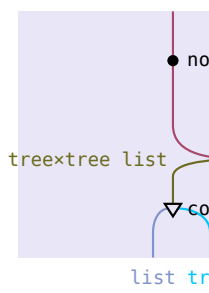
|   |                                                           |                                                                                       |
|---|-----------------------------------------------------------|---------------------------------------------------------------------------------------|
| 4 | = nonsingle (init inits, 1) (list(mct) $\times$ mct) snoc | $\langle R, S \rangle (U \times V) = \langle R U, S V \rangle$                        |
| 5 | = nonsingle (init inits list(mct), mct) snoc              | col = inits list(mct)                                                                 |
| 6 | = nonsingle (init col, mct) snoc                          | nonsingle $\langle U, V \rangle =$ nonsingle $\langle U, \text{nonsingle } V \rangle$ |

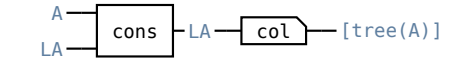
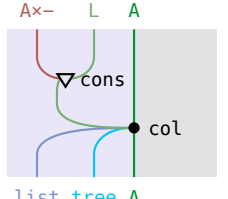
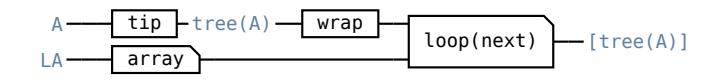
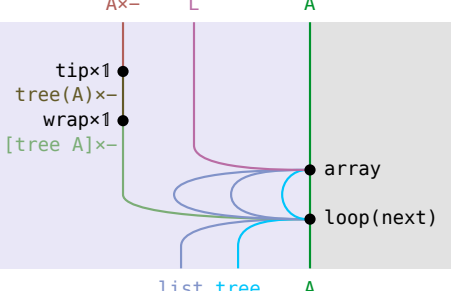
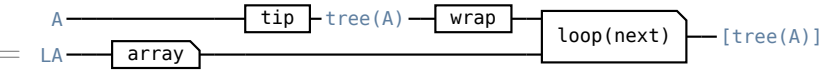
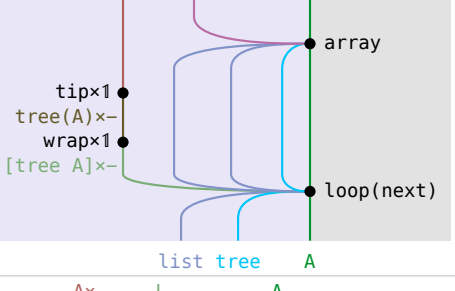
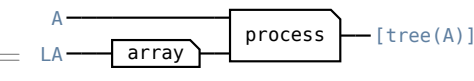
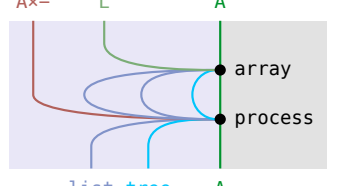


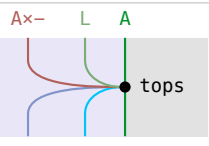
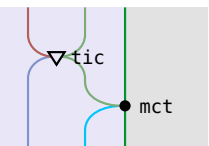
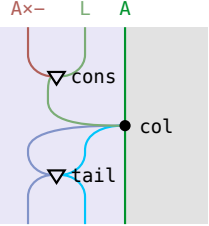
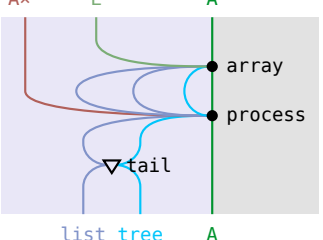
|   |                                                                 |                                                                                                                         |
|---|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 7 | = nonsingle (init col, nonsingle mct) snoc                      | (9.7) nonsingle mct = nonsingle (init col, tail row) mix                                                                |
| 8 | = nonsingle (init col, nonsingle (init col, tail row) mix) snoc | nonsingle $\langle U, V \rangle =$ nonsingle $\langle U, \text{nonsingle } V \rangle$                                   |
| 9 | = nonsingle (init col, (init col, tail row) mix) snoc           | $\langle \text{init col}, V \rangle \langle \pi_1, M \rangle = \langle \text{init col}, (\text{init col}, V) M \rangle$ |

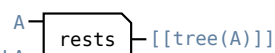
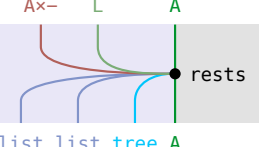
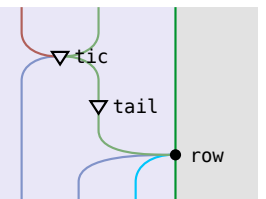
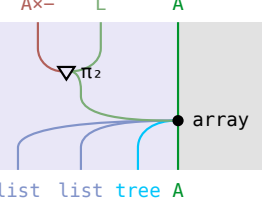


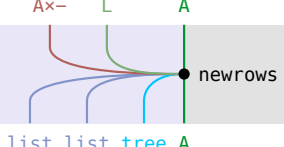
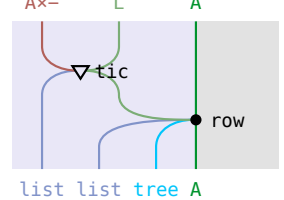
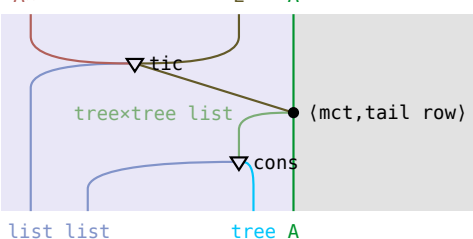
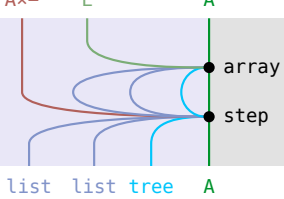
|    |                                                                           |                                                 |
|----|---------------------------------------------------------------------------|-------------------------------------------------|
| 10 | = nonsingle (init col, tail row) $\langle \pi_1, \text{mix} \rangle$ snoc | next = $\langle \pi_1, \text{mix} \rangle$ snoc |
| 11 | = nonsingle (init col, tail row) next                                     |                                                 |

| nonsingle row = nonsingle (mct,tail row) cons (9.10)                                                                                                                                                                                                                        |                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| on a list of two or more elements, the row is mct of the whole list consed onto the row of its tail                                                                                                                                                                         |                                                               |
|    <p>(1) (2) (3)</p>    |                                                               |
| 1 nonsingle row                                                                                                                                                                                                                                                             | row $\triangleq$ tails list(mct)                              |
| 2 $\triangleq$ nonsingle tails list(mct)                                                                                                                                                                                                                                    | nonsingle tails M = nonsingle (1,tail tails)<br>cons M        |
| 3 = nonsingle (1,tail tails) cons list(mct)                                                                                                                                                                                                                                 | cons list(f) = (f $\times$ list(f)) cons                      |
|    <p>(4) (5) (6)</p> |                                                               |
| 4 = nonsingle (1,tail tails) (mct $\times$ list(mct)) cons                                                                                                                                                                                                                  | $\langle R,S \rangle (U \times V) = \langle R U, S V \rangle$ |
| 5 = nonsingle (mct,tail tails list(mct)) cons                                                                                                                                                                                                                               | row = tails list(mct)                                         |
| 6 = nonsingle (mct,tail row) cons                                                                                                                                                                                                                                           |                                                               |

| cons col = (1 $\times$ array) process (9.9)                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| the column of a consed onto x is what process makes of a and the array of x                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                          |
| circuit                                                                                                                                                                                                                                                                                                                                                                                                                                          | Hinze–Marsden                                                                                            |
|                                                                                                                                                                                                                                                                                                                                                               |  <p>list tree A</p> |
|  <p>= Exercise 9.13 with <math>k \triangleq</math> cons col, <math>g \triangleq</math> tip wrap, <math>h \triangleq</math> row, <math>f \triangleq</math> next: its two equations are (9.8)<br/>— (9.8) nonsingle col = nonsingle (init col,tail row) next — at <math>a:[b]</math><br/>and at <math>a:(u++[b])</math>, with <math>[a]</math> col=[tip(a)]</p> |  <p>list tree A</p> |
|  <p>= products</p>                                                                                                                                                                                                                                                                                                                                            |  <p>list tree A</p> |
|  <p>= definition of process</p>                                                                                                                                                                                                                                                                                                                               |  <p>list tree A</p> |

| <b>tops = (1 × array) process tail</b> <span style="float: right;">(9.3c)</span><br>the trees at the top of the new rows are what <b>process</b> makes of <b>a</b> and the array of <b>x</b> , less its first |                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| circuit                                                                                                                                                                                                       | Hinze–Marsden                                                                        |
|                                                                                                                              |   |
| $= \begin{array}{c} A \\ LA \end{array} \text{cons} \text{LA} \text{inits} \text{tail} \text{LA} \text{list(mct)} \text{tree(A)}$ <p>definition of tops<math>\triangleq</math>tic list(mct), our name</p>     |   |
| $= \begin{array}{c} A \\ LA \end{array} \text{cons} \text{LA} \text{col} \text{tail} \text{tree(A)}$ <p>tail list(f)=list(f) tail; definition of col</p>                                                      |   |
| $= \begin{array}{c} A \\ LA \end{array} \text{array} \text{process} \text{tail} \text{tree(A)}$ <p>(9.9) — (9.9) cons col = (1 × array) process</p>                                                           |  |

| <b>rests = <math>\pi_2</math> array</b> <span style="float: right;">(9.3d)</span><br>the rest of each new row is the array of <b>x</b>                                                                               |                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| circuit                                                                                                                                                                                                              | Hinze–Marsden                                                                         |
|                                                                                                                                   |  |
| $= \begin{array}{c} A \\ LA \end{array} \text{cons} \text{LA} \text{inits} \text{tail} \text{LA} \text{list(tail row)} \text{tree(A)}$ <p>definition of rests<math>\triangleq</math>tic list(tail row), our name</p> |  |
| $= \begin{array}{c} A \\ LA \end{array} \text{array} \text{tree(A)}$ <p>tic list(tail)=<math>\pi_2</math> inits; definition of array</p>                                                                             |  |

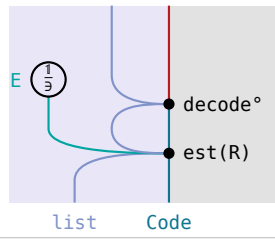
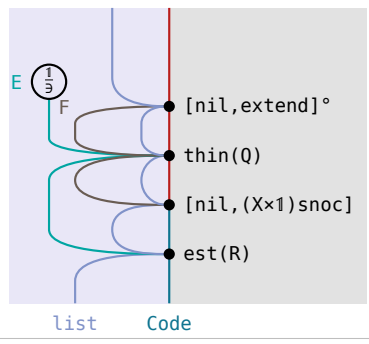
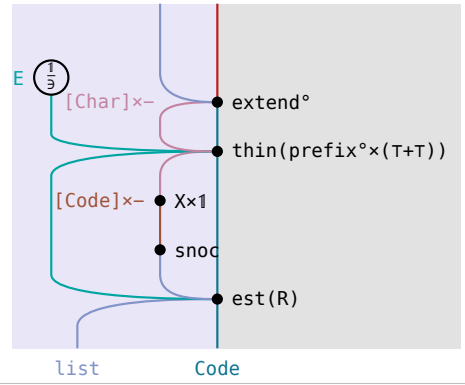
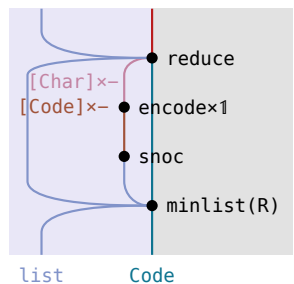
| $\text{newrows} = (1 \times \text{array}) \text{ step}$ <span style="float: right;">(9.3e)</span><br>the rows of the array of $\mathbf{a}:\mathbf{x}$ below its first are what <b>step</b> makes of $\mathbf{a}$ and the array of $\mathbf{x}$                                                                                                           |                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| circuit                                                                                                                                                                                                                                                                                                                                                  | Hinze–Marsden                                                                         |
|                                                                                                                                                                                                                                                                         |    |
| $= \text{LA} \rightarrow \text{cons} \rightarrow \text{LA} \rightarrow \text{inits} \rightarrow \text{tail} \rightarrow [\text{LA}] \rightarrow \text{list}(\text{row}) \rightarrow [[\text{tree}(\text{A})]]$ <p>definition of <math>\text{newrows} \triangleq \text{tic list}(\text{row})</math>, our name</p>                                         |    |
| $= \text{LA} \rightarrow \text{cons} \rightarrow \text{LA} \rightarrow \text{inits} \rightarrow \text{tail} \rightarrow [\text{LA}] \rightarrow \text{list}(\text{mct, tail row cons}) \rightarrow [[\text{tree}(\text{A})]]$ <p>(9.10) — (9.10) nonsingle row = nonsingle (mct, tail row)<br/> cons — on the non-singleton tic lists</p>                |    |
| $= \text{LA} \rightarrow \text{tops} \rightarrow \text{rests} \rightarrow \text{zip} \rightarrow [\text{tree}(\text{A}) \times [\text{tree}(\text{A})]] \rightarrow \text{list}(\text{cons}) \rightarrow [[\text{tree}(\text{A})]]$ <p><math>\text{list}(f,g) = (\text{list}(f), \text{list}(g))</math> zip; products; definitions of tops and rests</p> |   |
| $= \text{LA} \rightarrow \text{array} \rightarrow \text{step} \rightarrow [[\text{tree}(\text{A})]]$ <p><math>\text{tops} = (1 \times \text{array})</math> process tail and rests = <math>\pi_2</math> array;<br/> products; definition of step</p>                                                                                                      |  |

| $\text{cons array} = (1 \times \text{array}) \text{ addcol}$ <span style="float: right;">(9.3f)</span>                                                             |                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| the array of <b>a</b> consed onto <b>x</b> is what <b>addcol</b> makes of <b>a</b> and the array of <b>x</b>                                                       |                         |
| circuit                                                                                                                                                            | Hinze–Marsden           |
| <p><math>A</math> and <math>LA</math> inputs to <math>\text{cons}</math>, followed by <math>\text{array}</math>, resulting in <math>[[\text{tree}(A)]]</math>.</p> | <p>list list tree A</p> |
| $=$ <p>definition of <b>array</b></p>                                                                                                                              | <p>list list tree A</p> |
| $=$ <p><math>\text{cons inits} = (\pi_1 \text{ wrap, tic}) \text{ cons}</math>, abbreviating <math>\text{cons inits tail}</math> by <b>tic</b></p>                 | <p>list list tree A</p> |
| $=$ <p><math>\text{cons list}(f) = (f \times \text{list}(f)) \text{ cons}</math>; definition of <b>newrows</b></p>                                                 | <p>list list tree A</p> |
| $=$ <p><math>\text{wrap row} = \text{tip wrap}</math></p>                                                                                                          | <p>list list tree A</p> |
| $=$ <p><math>\text{newrows} = (1 \times \text{array}) \text{ step}</math>; products; definition of <b>addcol</b></p>                                               | <p>list list tree A</p> |

## §9.4. Data compression

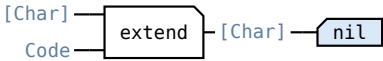
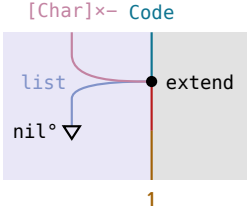
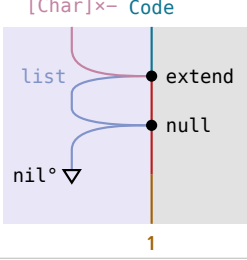
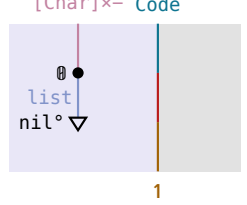
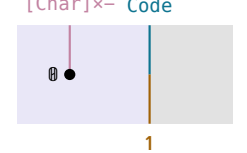
(9.4a)

|    | name                    | type                                                           | definition                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | meaning                                                                                                                                    |
|----|-------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | String                  | Type                                                           | $\text{String} \triangleq [\text{Char}]$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | a string is a snoc-list of characters                                                                                                      |
| 2  | LA                      | Type                                                           | $\text{LA} \triangleq [\text{A}]$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | a non-empty snoc-list: one value, then any number more appended                                                                            |
| 3  | Code                    | Type                                                           | $\text{Code} \triangleq \text{sym}(\text{Char}) \mid \text{ptr}(\text{String}, \text{String})$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | a code is one character, or a pointer back to a repeated piece                                                                             |
| 4  | extend                  | $\text{String} \times \text{Code} \rightarrow [\text{Char}]$   | $\text{extend} \triangleq \text{extend} \ , \ \text{extend}((\text{xs}, \text{sym}(\text{a})), \text{x}) = \text{x} = \text{snoc}(\text{xs}, \text{a}) \ ,$<br>$\text{extend}((\text{xs}, \text{ptr}(\text{ys}, \text{zs})), \text{x}) = \text{x} = \text{cat}(\text{xs}, \text{zs}) \wedge \text{zs} \neq [] \wedge (\text{init}^+)$<br>$(\text{cat}(\text{ys}, \text{zs}), \text{cat}(\text{xs}, \text{zs}))$                                                                                                                                                                                                                                                                                                                                                                                     | a symbol appends its character; a pointer appends $\text{zs}$ when $\text{ys} \# \text{zs}$ is a proper prefix of $\text{xs} \# \text{zs}$ |
| 5  | decode                  | $[\text{Code}] \rightarrow [\text{Char}]$                      | $\text{decode} \triangleq ([\text{nil}, \text{extend}])$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | rebuild the string from its codes, left to right; $\text{H} = \text{decode}^\circ$                                                         |
| 6  | bytes                   | $\text{Code} \rightarrow \mathbb{N}$                           | $\text{bytes}(\text{sym}(\text{a})) = \text{c} \ , \ \text{bytes}(\text{ptr}(\text{a}, \text{a}')) = \text{p}$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | a symbol costs $\text{c}$ , a pointer costs $\text{p}$                                                                                     |
| 7  | size                    | $[\text{Code}] \rightarrow \mathbb{N}$                         | $([\text{nil}, \text{distr} \ [1 \times \text{c}, 1 \times \text{p}] \text{ plus}]) = \text{size} \ , \ \text{size}(\text{wrap}(\text{a})) = 0 \ ,$<br>$\text{size}(\text{snoc}(\text{cs}, \text{e})) = \text{size}(\text{cs}) + \text{bytes}(\text{e})$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | the total cost of a code sequence                                                                                                          |
| 8  | R                       | $[\text{Code}] \rightarrow [\text{Code}]$                      | $\text{R} = \text{size} \leq \text{size}^\circ \ , \ \text{R}(\text{xs}, \text{ys}) \iff \text{size}(\text{xs}) \leq \text{size}(\text{ys})$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | $\text{u}$ costs no more than $\text{v}$                                                                                                   |
| 9  | prefix                  | $[\text{Char}] \rightarrow [\text{Char}]$                      | $\text{xs prefix ys} \iff (\text{prefix}^\circ)(\text{ys}, \text{xs})$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | $\text{ys}$ is a prefix of $\text{x}$                                                                                                      |
| 10 | $(\text{T} + \text{T})$ | $\text{Code} \rightarrow \text{Code}$                          | $(\text{T} + \text{T}) = [\text{sym}, \text{ptr}]^\circ \ (\text{T} + \text{T}) \ [\text{sym}, \text{ptr}] \ , \ \text{sym}(\text{a}) \ (\text{T} + \text{T}) \ \text{sym}(\text{a}') \iff$<br>$\text{true} \ , \ \text{ptr}(\text{a}, \text{a}') \ (\text{T} + \text{T}) \ \text{ptr}(\text{a}'', \text{a}''') \iff \text{true} \ , \ (\forall \text{a}. \ \forall \text{a}'. \ \text{e} =$<br>$\text{sym}(\text{a}) \implies \text{f} = \text{sym}(\text{a}') \implies \text{false}) \wedge (\forall \text{a}. \ \forall \text{a}'. \ \forall \text{a}'''. \ \forall \text{a}'''. \ \text{e} =$<br>$\text{ptr}(\text{a}, \text{a}') \implies \text{f} = \text{ptr}(\text{a}'', \text{a}''') \implies \text{false}) \implies (\text{e} \ (\text{T} + \text{T}) \ \text{f} \iff$<br>$\text{false})$ | any symbol to any symbol, any pointer to any pointer                                                                                       |
| 11 | Q                       | $\text{F}[\text{Char}] \rightarrow \text{F}[\text{Char}]$      | $\text{Q} = \text{F}((\text{T} + \text{T}), \text{prefix}^\circ) \ , \ \text{l}(\text{val}) \ \text{Q} \ \text{l}(\text{val}') \iff \text{true} \ , \ \text{r}(\text{p}) \ \text{Q} \ \text{r}(\text{q})$<br>$\iff (\text{prefix}^\circ)(\pi_1(\text{p}), \pi_1(\text{q})) \wedge (\text{T} + \text{T})(\pi_2(\text{p}), \pi_2(\text{q})) \ , \ (\forall \text{val}. \ \forall \text{val}'. \$<br>$\text{u} = \text{l}(\text{val}) \implies \text{v} = \text{l}(\text{val}') \implies \text{false}) \wedge (\forall \text{p}. \ \forall \text{q}. \ \text{u} = \text{r}(\text{p}) \implies \text{v} =$<br>$\text{r}(\text{q}) \implies \text{false}) \implies (\text{u} \ \text{Q} \ \text{v} \iff \text{false})$                                                                                   | compare two decompositions by the prefix order on the strings and $\text{U}$ on the codes                                                  |
| 12 | reduce                  | $[\text{Char}] \rightarrow [\text{String} \times \text{Code}]$ | $\text{reduce} \triangleq \text{reduce} \ , \ \text{q} \in \text{reduce}(\text{ws}) \iff \text{extend}(\text{q}, \text{ws})$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | the last code of a string: one symbol, or a pointer to its longest repeated tail                                                           |

| circuit                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Hinze–Marsden                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| $(\mu X : \frac{[nil, extend]^{\circ}}{\exists} \text{thin}(Q) P([nil, (X \times 1) snoc]) \text{est}(R)) \sqsubseteq \frac{decode^{\circ}}{\exists} \text{est}(R) ,$ $\text{encode} = (\text{null} \rightarrow nil, \text{reduce } list((\text{encode} \times 1) snoc) \text{minlist}(R))$ <p>a smallest code sequence decoding to the given string is built from the right, each step emitting the last character as a symbol or ending with a pointer back into what has already been decoded</p> <p>the specification — <math>decode \triangleq \lambda [nil, extend]</math></p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | <p><b>Hinze–Marsden</b></p>  |
| $[Char] \rightarrow \frac{1}{\exists} P[Char] \rightarrow E([nil, extend]^{\circ}) \rightarrow \text{thin}(Q) \rightarrow PF[Char] \rightarrow P([nil, (X \times 1) snoc]) \rightarrow P[Code] \rightarrow \text{est}(R) \rightarrow [Code]$ <p>at <math>Q \triangleq F(T+T, prefix^{\circ}) = 1 + (prefix^{\circ} \times (T+T))</math>, the two <math>T</math> on symbols and on pointers.<br/> Monotonicity <math>F(R) \alpha \sqsubseteq \alpha R</math> is routine, the two costs being constants: <math>(T+T)[c, p] = [c, p]</math>.</p> <p>Proposition 9.4 splits the thinning condition in two — <math>F(T+T, R) \alpha \sqsubseteq \alpha R</math>, and <math>decode</math> <math>prefix \sqsubseteq R^{\circ} decode</math>, for which <math>decode \text{init} \sqsubseteq R^{\circ} decode</math> is enough: dropping the last character of the output shortens or removes the last code element and never raises the cost. So between a symbol and a pointer nothing can be decided in advance, and between two pointers the longer match wins</p> |                              |
| $[Char] \rightarrow \frac{1}{\exists} P[Char] \rightarrow E(extend^{\circ}) \rightarrow \text{thin}(prefix^{\circ} \times (T+T)) \rightarrow P([Char] \times Code) \rightarrow P((X \times 1) snoc) \rightarrow P[Code] \rightarrow \text{est}(R) \rightarrow [Code]$ <p>Proposition 9.1: <math>nil</math> and <math>extend</math> have disjoint ranges. The decompositions of one string are <math>\frac{extend^{\circ}}{\exists} (ws \# [a]) = \{(ws, \text{sym } a)\} \cup \{(xs, \text{ptr } (ys, zs)) \mid xs \# zs = ws \# [a], ys \# zs \text{ a proper prefix of } ws\}</math> — take the last character as a symbol, or end with a pointer</p>                                                                                                                                                                                                                                                                                                                                                                                                        |                             |
| $[Char] \rightarrow \text{reduce} \rightarrow [[Char] \times Code] \rightarrow list((\text{encode} \times 1) snoc) \rightarrow [[Code]] \rightarrow \text{minlist}(R) \rightarrow [Code]$ <p><math>\text{reduce}</math> implements <math>\frac{extend^{\circ}}{\exists} \text{thin}(prefix^{\circ} \times (T+T))</math>: thinning leaves at most two, the symbol and the pointer of the longest repeated tail <math>lrt</math>. Again exponential; the book gives no tabulation for it</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                            |

### §9.4.1. Derivation

| $Q \vdash F(\text{decode}^\circ) [\text{nil}, \text{snoc}] \sqsubseteq F(\text{decode}^\circ) [\text{nil}, \text{snoc}] R$ (9.4.1a)                                                                                                                                                                                                                                                                                                                                                                                                       |               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| every code sequence built from a decomposition $Q$ puts above a given one — a code element of the same kind, a longer front string — costs at least as much as one built from the given one                                                                                                                                                                                                                                                                                                                                               |               |
| circuit                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Hinze–Marsden |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |               |
| $=$ <p> <math>Q \triangleq 1 + (\text{prefix}^\circ \times (T+T))</math> is <math>F(T+T, \text{prefix}^\circ) - l(\text{val}) \ Q \ l(\text{val}') \iff \text{true}, r(p) \ Q \ r(q) \iff (\text{prefix}^\circ)(\pi_1(p), \pi_1(q)) \wedge (T+T)(\pi_2(p), \pi_2(q)), (\forall \text{val}. \forall \text{val}'. u = l(\text{val}) \implies v = l(\text{val}') \implies \text{false}) \wedge (\forall p. \forall q. u = r(p) \implies v = r(q) \implies \text{false})</math><br/> <math>\implies (u \ Q \ v \iff \text{false})</math> </p> |               |
| $=$ <p>the bifunctor <math>F</math> preserves composition: <math>F(U, \text{prefix}^\circ) F(1, \text{decode}^\circ) = F(U, \text{prefix}^\circ \text{decode}^\circ)</math></p>                                                                                                                                                                                                                                                                                                                                                           |               |
| $\sqsubseteq$ <p>Proposition 9.4's second condition <math>\text{prefix}^\circ \text{decode}^\circ \sqsubseteq \text{decode}^\circ R</math>: dropping the last character drops the last <b>sym</b>, or shortens or drops the last pointer. Proved at <b>prefix</b> directly by induction, not through <b>init</b></p>                                                                                                                                                                                                                      |               |
| $=$ <p><math>F</math> preserves composition: <math>F(U, \text{decode}^\circ R) = F(1, \text{decode}^\circ) F(U, R)</math></p>                                                                                                                                                                                                                                                                                                                                                                                                             |               |
| $\sqsubseteq$ <p>Proposition 9.4's first condition <math>F(T+T, R) \alpha \sqsubseteq \alpha R</math>, left as an exercise in the book: <b>snoc</b> adds the cost of the last element to both sides, and <math>[c, p](T+T) = [c, p]</math></p>                                                                                                                                                                                                                                                                                            |               |

| <div> <div>extend nil<sup>°</sup> = 0</div> <div>(9.4.1b)</div> </div> <div>no string is both a result of <b>extend</b> and the result [] of <b>nil</b></div>                                                       |                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| circuit                                                                                                                                                                                                             | Hinze–Marsden                                                                        |
|                                                                                                                                    |   |
| <div> <div> <div>[Char]</div> <div>Code</div> </div> <div>extend</div> <div> <div>null</div> <div>[Char]</div> <div>nil</div> </div> </div> <div>nil returns only [], so nil<sup>°</sup>=null nil<sup>°</sup></div> |   |
| <div> <div> <div>[Char]</div> <div>Code</div> </div> <div>0</div> <div>[Char]</div> <div>nil</div> </div> <div>extend null=0: a symbol ends the string with a character, a pointer with its non-empty ZS</div>      |   |
| <div> <div> <div>[Char]</div> <div>Code</div> </div> <div>0</div> </div> <div>0 composed with anything is 0</div>                                                                                                   |  |